

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH
BỘ MÔN HỆ THỐNG THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

XÂY DỰNG GIẢI PHÁP GOM CỤM TÌNH TRẠNG
GIAO THÔNG Ở TP.HCM

Ngành: Khoa học Máy tính

Mã đồ án: HK253-DATN-055

GVHD: PGS.TS. Trần Minh Quang
ThS. Bùi Tiến Đức

GVPB: PGS.TS. Võ Thị Ngọc Châu

SVTH: Trần Quang Huy — 2211288

TP. HỒ CHÍ MINH, THÁNG 09 NĂM 2026

LỜI CẢM ƠN

Trước tiên, em xin gửi lời cảm ơn chân thành tới PGS.TS. Trần Minh Quang và ThS. Bùi Tiến Đức — những người đã tận tình hướng dẫn, định hướng và góp ý cho em trong suốt quá trình thực hiện đồ án. Nhờ sự chỉ bảo của các thầy, em không chỉ hoàn thành được các mục tiêu nghiên cứu mà còn học được cách tiếp cận một bài toán khoa học một cách có hệ thống: từ khảo sát công trình liên quan, xây dựng và kiểm chứng giả thuyết, cho đến hiện thực và đánh giá sản phẩm.

Em cũng xin gửi lời cảm ơn Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách Khoa — ĐHQG TP.HCM đã tạo môi trường học tập và nghiên cứu; quý thầy cô trong bộ môn Hệ thống Thông tin đã truyền đạt những kiến thức nền tảng mà em vận dụng xuyên suốt đồ án. Cuối cùng, em xin cảm ơn gia đình và bạn bè đã luôn đồng hành, động viên em trong suốt quá trình học tập và thực hiện đồ án tốt nghiệp.

Mặc dù đã nỗ lực hết sức, đồ án không thể tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của quý thầy cô để báo cáo được hoàn thiện hơn.

TP.HCM, tháng 09 năm 2026

Sinh viên thực hiện

Trần Quang Huy

TÓM TẮT BÁO CÁO

Ùn tắc giao thông là một trong những vấn đề cấp bách nhất của Thành phố Hồ Chí Minh. Đề tài “*Xây dựng giải pháp gom cụm tình trạng giao thông ở TP.HCM*” hướng tới việc nhận diện tự động các trạng thái giao thông điển hình của thành phố bằng kỹ thuật gom cụm dữ liệu, đồng thời đưa kết quả phân tích tới tay người tham gia giao thông thông qua một hệ thống ứng dụng hoàn chỉnh. Hệ thống gồm bốn thành phần:

- (i) pipeline thu thập và xử lý dữ liệu giao thông đa nguồn (TomTom Traffic API, OpenStreetMap, Mapbox, Open-Meteo);
- (ii) pipeline học máy so sánh năm thuật toán gom cụm (K-Means, MiniBatchK-Means, DBSCAN, GMM, Agglomerative Clustering) bằng bộ chỉ số Silhouette, Davies–Bouldin, Calinski–Harabasz, ARI, NMI;
- (iii) ứng dụng di động **TraffiGo** trên nền Android cho người dùng đi đường;
- (iv) **server Machine Learning** (FastAPI) cung cấp API dự đoán tốc độ lưu thông, mức độ kẹt xe, rủi ro sự cố và gợi ý tuyến đường theo từng đoạn đường.

Kết quả thực nghiệm cho thấy mô hình K-Means với $k = 6$ trên nhánh Feature Selection đạt chất lượng gom cụm tốt nhất (Silhouette 0,4183; Davies–Bouldin 0,8063; Calinski–Harabasz 147574,6) và cho phép diễn giải sáu trạng thái giao thông có ý nghĩa thực tiễn, từ “tắc tuyệt đối/bất thường” đến “thông thoáng hoàn toàn”. Mô hình này được xuất ở dạng JSON và nhúng trực tiếp vào ứng dụng di động, giúp tô màu sáu mức trạng thái giao thông trên bản đồ trong thời gian thực mà không phụ thuộc máy chủ. Song song, mô hình Random Forest dự đoán tốc độ lưu thông được huấn luyện trên khoảng 149.000 mẫu dữ liệu thu thập trong giai đoạn 09/03–15/04/2026, đạt MAE 2,19 km/h (so với baseline 8,25 km/h) và hệ số xác định $R^2 = 0,919$; mô hình được triển khai qua bộ REST API `/api/ml/*` và tích hợp hai chiều với ứng dụng: dự báo chi tiết theo đoạn đường và gợi ý tuyến đường tối ưu. Server chính thức hiện chạy trên Render Cloud tại <https://traffigo-server.onrender.com>; ứng dụng Android kết nối tới dịch vụ này và hiển thị chip xanh “Server ML: đang hoạt động” khi kiểm tra `/health` thành công. Ứng dụng đã vượt qua toàn bộ ca kiểm thử đơn vị và ca kiểm thử chức năng trên thiết bị giả lập.

Từ khóa: gom cụm dữ liệu, học máy không giám sát, K-Means, dự đoán giao thông, ứng dụng Android, FastAPI.

ABSTRACT

Traffic congestion is one of the most pressing problems in Ho Chi Minh City. This project, “*Building a traffic condition clustering solution for Ho Chi Minh City*”, aims to automatically identify the typical traffic states of the city using data clustering techniques, and to deliver the analysis results to road users through a complete application system. The system comprises four components:

- (i) a multi-source traffic data collection and processing pipeline (TomTom Traffic API, OpenStreetMap, Mapbox, Open-Meteo);
- (ii) a machine learning pipeline that compares five clustering algorithms (K-Means, MiniBatchKMeans, DBSCAN, GMM, Agglomerative Clustering) using Silhouette, Davies–Bouldin, Calinski–Harabasz, ARI and NMI metrics;
- (iii) **TraffiGo**, an Android application for road users;
- (iv) a **Machine Learning server** (FastAPI) exposing APIs that predict traffic speed, congestion level and incident risk per road segment.

Experiments show that K-Means with $k = 6$ on the Feature Selection branch achieves the best clustering quality (Silhouette 0.4183; Davies–Bouldin 0.8063; Calinski–Harabasz 147574.6) and yields six practically interpretable traffic states, from “extreme congestion/anomaly” to “free flow”. This model is exported as JSON and embedded directly in the mobile application, coloring six real-time traffic states on the map without depending on a server. In parallel, a Random Forest speed-prediction model trained on approximately 149,000 samples collected between 2026-03-09 and 2026-04-15 achieves MAE 2.19 km/h (versus an 8.25 km/h baseline) with $R^2 = 0.919$; the model is deployed through REST APIs (/api/ml/*) and integrated with the application for per-segment forecasting and route recommendation. The production server currently runs on Render Cloud at <https://traffigo-server.onrender.com>; the Android application connects to this service and shows a green “Server ML: online” status chip when the /health check succeeds. The application passed all unit tests and functional test cases on an Android emulator.

Keywords: data clustering, unsupervised learning, K-Means, traffic prediction, Android application, FastAPI.

MỤC LỤC

LỜI CẢM ƠN	1
TÓM TẮT BÁO CÁO	2
ABSTRACT	3
DANH MỤC HÌNH	8
DANH MỤC BẢNG	9
DANH MỤC TỪ VIẾT TẮT	11
1 GIỚI THIỆU	12
1.1 Động cơ của đề tài	12
1.2 Mục tiêu	12
1.3 Phạm vi	13
1.4 Ý nghĩa	13
1.4.1 Ý nghĩa thực tiễn	13
1.4.2 Ý nghĩa khoa học	14
1.5 Cấu trúc báo cáo	14
2 KIẾN THỨC VÀ CÔNG NGHỆ NỀN TẢNG	15
2.1 Bài toán nhận diện trạng thái giao thông và hướng tiếp cận không giám sát	15
2.2 Góm cụm dữ liệu	15
2.2.1 Nhóm phân hoạch: K-Means	16
2.2.2 Nhóm phân hoạch: MiniBatchKMeans	16
2.2.3 Nhóm mật độ: DBSCAN	17
2.2.4 Nhóm mô hình xác suất: Gaussian Mixture Model	17
2.2.5 Nhóm phân cấp: Agglomerative Clustering	17
2.2.6 So sánh các thuật toán	18
2.2.7 Đánh giá mức phù hợp với bài toán giao thông	18
2.3 Chuẩn hóa dữ liệu và lựa chọn thuộc tính	19
2.3.1 Chuẩn hóa	19

2.3.2	Lựa chọn thuộc tính	19
2.4	Các chỉ số đánh giá chất lượng gom cụm	19
2.4.1	Chỉ số nội bộ (internal)	19
2.4.2	Chỉ số ngoại bộ (external)	20
2.5	Nguồn dữ liệu giao thông và thời tiết	20
2.6	Công nghệ phát triển ứng dụng di động	21
2.7	Công nghệ xây dựng server Machine Learning	21
2.8	Kiến trúc ứng dụng Android và mô hình thực thi	22
2.9	Kết chương	23
3	CÔNG TRÌNH LIÊN QUAN	24
3.1	Tổng quan các hướng tiếp cận	24
3.2	Ba công trình nghiên cứu tiêu biểu	24
3.2.1	Điểm khác biệt giữa các công trình và đề tài	24
3.3	Các hệ thống và công cụ thực tế liên quan	25
3.4	Nhận xét chung và khoảng trống nghiên cứu	25
3.5	Kết chương	26
4	HỆ THỐNG ĐỀ XUẤT	27
4.1	Mô tả hệ thống	27
4.2	Yêu cầu	28
4.2.1	Yêu cầu chức năng	28
4.2.2	Yêu cầu phi chức năng	29
4.2.3	Yêu cầu dữ liệu	29
4.3	Kết chương	30
5	PHÂN TÍCH VÀ THIẾT KẾ	31
5.1	Phân tích	31
5.1.1	Mô hình hóa quy trình nghiệp vụ	31
5.1.2	Mô hình hóa tương tác	31
5.1.3	Sơ đồ hoạt động và tuần tự của luồng tìm đường	32
5.2	Giải pháp công nghệ	33
5.2.1	Front-end (ứng dụng di động)	33
5.2.2	Back-end (server ML)	35
5.3	Thiết kế	36
5.3.1	Kiến trúc hệ thống	36
5.3.2	Thiết kế cơ sở dữ liệu	36
5.3.3	Thiết kế chi tiết bên trong	37
5.3.4	Thiết kế giao diện	41
5.4	Kết chương	41

6	HIỆN THỰC VÀ KIỂM THỬ	42
6.1	Hiện thực	42
6.1.1	Pipeline dữ liệu	42
6.1.2	Kết quả lựa chọn thuộc tính và gom cụm	42
6.1.3	Xuất mô hình và nhúng lên ứng dụng	44
6.1.4	Server Machine Learning	44
6.1.5	Tích hợp vào ứng dụng di động	45
6.1.6	Cập nhật dữ liệu live định kỳ phía server	46
6.1.7	Triển khai server chính thức trên Render Cloud	46
6.1.8	Kết quả ứng dụng	47
6.2	Ứng dụng kết quả gom cụm vào ứng dụng TraffiGo	49
6.2.1	Tô màu trạng thái giao thông thời gian thực (on-device)	49
6.2.2	Định tuyến né khu vực kẹt	53
6.2.3	Chấm điểm sức khỏe tuyến A–F	54
6.2.4	Dự báo và gán cụm phía server (thẻ Dự báo ML)	55
6.2.5	Tự động chuyển tuyến theo đề xuất ML	55
6.2.6	Lớp thành thật dữ liệu	55
6.2.7	Kịch bản người dùng cuối	55
6.3	Kiểm thử	56
6.3.1	Môi trường kiểm thử	56
6.3.2	Đo hiệu năng	56
6.3.3	Kiểm thử đơn vị	56
6.3.4	Kiểm thử chức năng	57
6.3.5	Lỗi phát hiện và khắc phục	57
6.4	Kết chương	58
7	ĐÁNH GIÁ HỆ THỐNG	59
7.1	Đánh giá theo yêu cầu	59
7.2	So sánh với các giải pháp hiện có	59
7.3	Ưu điểm	59
7.4	Hạn chế	60
7.5	Phân tích rủi ro triển khai	60
7.6	Kiểm thử hồi quy sau khắc phục lỗi	61
7.7	Kết chương	61
8	TỔNG KẾT	62
8.1	Kết quả đạt được	62
8.2	Hướng phát triển	62
	KẾ HOẠCH THỰC HIỆN ĐỒ ÁN TỐT NGHIỆP	64

TÀI LIỆU THAM KHẢO	65
A HƯỚNG DẪN CHẠY SERVER ML	67
B ĐẶC TRƯNG CÁC MÔ HÌNH	68

DANH MỤC HÌNH

2.1	Minh họa hai bước gán cụm và cập nhật tâm của K-Means trên dữ liệu hai chiều.	16
5.1	Sơ đồ khối quy trình nghiệp vụ của pipeline dữ liệu và học máy.	32
5.2	Sơ đồ Use Case tổng thể của hệ thống.	33
5.3	Sơ đồ hoạt động của luồng tìm đường kết hợp dự báo ML.	36
5.4	Sơ đồ tuần tự luồng tìm đường kết hợp dự báo ML.	37
5.5	Kiến trúc tổng thể ba tầng của hệ thống.	38
5.6	Cấu trúc dữ liệu người dùng trên Firebase Realtime Database.	38
6.1	So sánh Silhouette Score của năm thuật toán trên nhánh Feature Selection.	43
6.2	Phản hồi trạng thái hệ thống cho người dùng.	47
6.3	Luồng khởi động và đăng nhập của ứng dụng.	48
6.4	Màn hình Trang chủ của TraffiGo.	49
6.5	Bản đồ Tình trạng giao thông: các đoạn tô màu theo cụm K-Means.	50
6.6	Bottom sheet chi tiết đoạn với thẻ Dự báo ML từ server.	51
6.7	Luồng chọn điểm đi — điểm đến.	52
6.8	Thông tin tuyến: thời gian, quãng đường, cảnh báo tắc, sức khỏe tuyến, ước tính nhiên liệu.	53
6.9	Tin tức giao thông và tìm địa điểm lân cận.	54
6.10	Hồ sơ, lịch sử lộ trình và bản đồ ngoại tuyến.	54

DANH MỤC BẢNG

2.1	So sánh đặc tính các thuật toán gom cụm khảo sát.	18
2.2	Các nguồn dữ liệu của đề tài.	21
2.3	So sánh các phương án framework cho server ML.	22
3.1	Đối chiếu phạm vi thực thi giữa các công trình và đề tài.	25
3.2	Đối chiếu các giải pháp liên quan với đề tài.	25
4.1	Yêu cầu chức năng của hệ thống.	28
4.2	Yêu cầu phi chức năng của hệ thống.	29
5.1	Đặc tả UC-01: Đăng nhập và xác thực.	33
5.2	Đặc tả UC-02: Xem bản đồ giao thông.	34
5.3	Đặc tả UC-03: Tìm đường và né kẹt.	34
5.4	Đặc tả UC-04: Xem dự báo ML theo đoạn đường.	35
5.5	Đặc tả UC-09: Huấn luyện lại mô hình.	35
5.6	Mô tả đường dẫn dữ liệu và quy tắc truy cập Firebase.	39
5.7	Tóm tắt schema tập dữ liệu huấn luyện (40 cột, ví dụ các cột chính).	39
6.1	Phân bố tập dữ liệu theo ngữ cảnh thời gian ($n = 169.897$ mẫu).	42
6.2	Kết quả đánh giá gom cụm trên nhánh Feature Selection ($k = 6$).	43
6.3	So sánh hai mô hình K-Means tốt nhất từ hai nhánh thực nghiệm ($k = 6$).	44
6.4	Diễn giải sáu cụm giao thông (số liệu từ <code>cluster_model.json</code>).	44
6.5	Kết quả mô hình Random Forest dự đoán tốc độ lưu thông (cấu hình lite đã triển khai).	44
6.6	Các API Machine Learning của hệ thống.	45
6.7	Biến cấu hình bộ cập nhật dữ liệu live của server.	46
6.8	Ánh xạ từ kết quả nghiên cứu tới các điểm tích hợp trong ứng dụng.	50
6.9	Ví dụ tính đặc trưng cho một đoạn (25/60 km/h, 1,2 km).	52
6.10	Kết quả phân loại hai đoạn đối chiếu (suy luận bằng <code>cluster_model.json</code>).	52
6.11	Môi trường kiểm thử.	56
6.12	Kết quả đo hiệu năng server ML (10 lần đo mỗi endpoint).	56
6.13	Kết quả kiểm thử chức năng trên trình giả lập.	57
7.1	Phân tích rủi ro triển khai và phương án ứng phó.	60

8.1	Kế hoạch thực hiện đồ án tốt nghiệp (15 tuần).	64
-----	--	----

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Ý nghĩa
API	Application Programming Interface — giao diện lập trình ứng dụng
ARI	Adjusted Rand Index — chỉ số Rand hiệu chỉnh
BFS	Breadth-First Search — tìm kiếm theo chiều rộng
CH	Calinski–Harabasz Index
CSDL	Cơ sở dữ liệu
DBI	Davies–Bouldin Index
DTW	Dynamic Time Warping — xoắn thời gian động
EM	Expectation–Maximization — thuật toán kỳ vọng–cực đại hóa
FMI	Fowlkes–Mallows Index
GMM	Gaussian Mixture Model — mô hình hỗn hợp Gaussian
GPS	Global Positioning System — hệ định vị toàn cầu
HTTP	HyperText Transfer Protocol
ITS	Intelligent Transportation Systems — hệ thống giao thông thông minh
JWT	JSON Web Token
LOS	Level of Service — mức độ phục vụ giao thông
ML	Machine Learning — học máy
NMI	Normalized Mutual Information
OSM	OpenStreetMap — bản đồ mở mã nguồn mở
POI	Point of Interest — điểm quan tâm trên bản đồ
RF	Random Forest — rừng ngẫu nhiên
RMSE	Root Mean Square Error — sai số bình phương trung bình gốc
REST	Representational State Transfer
TSP	Transportation System Plan
TTS	Text-to-Speech — chuyển văn bản thành giọng nói
UC	Use Case — ca sử dụng
UI	User Interface — giao diện người dùng
UX	User Experience — trải nghiệm người dùng
V2X	Vehicle-to-Everything

CHƯƠNG 1

GIỚI THIỆU

1.1. Động cơ của đề tài

Trong những năm gần đây, tình trạng ùn tắc giao thông tại các đô thị lớn như Thành phố Hồ Chí Minh ngày càng trở nên nghiêm trọng. Theo thống kê của Sở Giao thông Vận tải thành phố, lượng phương tiện cá nhân đăng ký mới tiếp tục gia tăng trong khi quỹ đất dành cho hạ tầng giao thông gần như đã bão hòa, khiến các tuyến động mạch như Nguyễn Văn Linh, Xa lộ Hà Nội hay Võ Văn Kiệt thường xuyên ở trạng thái quá tải vào giờ cao điểm. Hệ lụy của tình trạng này không chỉ dừng ở việc mất thời gian di chuyển mà còn bao gồm chi phí nhiên liệu lãng phí, phát thải khí nhà kính tăng cao và năng suất lao động của toàn thành phố bị ảnh hưởng.

Để điều hành giao thông hiệu quả, cơ quan quản lý và chính người tham gia giao thông đều cần hiểu rõ *quy luật biến động* của dòng xe và phân loại được các *trạng thái giao thông* điển hình theo từng khu vực, từng thời điểm. Tuy nhiên, dữ liệu giao thông hiện đại thu thập từ các hệ thống GPS, cảm biến và nền tảng bản đồ số có đặc tính khổng lồ, đa chiều và biến đổi liên tục theo thời gian thực. Các phương pháp phân tích truyền thống hoặc thống kê mô tả đơn thuần gặp nhiều hạn chế trong việc bóc tách các đặc trưng phức tạp của mạng lưới giao thông đô thị, đặc biệt khi dữ liệu đến từ nhiều nguồn khác nhau với tần suất và độ tin cậy không đồng đều.

Kỹ thuật gom cụm dữ liệu (clustering) trong học máy không giám sát cung cấp một hướng tiếp cận phù hợp cho bài toán này: hệ thống có thể tự động nhóm các đoạn đường vào những *trạng thái giao thông* điển hình dựa trên sự tương đồng về tốc độ lưu thông, hạ tầng, thời tiết và ngữ cảnh thời gian, mà không cần gán nhãn thủ công. Từ đó, mỗi đoạn đường trên bản đồ được nhận diện thuộc “vùng kẹt”, “vùng thoáng” hay các trạng thái trung gian, làm cơ sở cho tô màu bản đồ, định tuyến né kẹt và dự đoán tình trạng giao thông.

Tuy nhiên, kết quả phân tích chỉ thực sự tạo giá trị khi được đưa tới tay người tham gia giao thông *đúng lúc, đúng nơi, ở dạng dễ hiểu*. Xuất phát từ thực tế đó, đề tài không dừng lại ở khâu nghiên cứu mô hình mà tiếp tục xây dựng một hệ thống ứng dụng hoàn chỉnh: ứng dụng di động TraffiGo dành cho người dùng đi đường và một server học máy cung cấp dịch vụ dự đoán, đóng vai trò cầu nối giữa kết quả gom cụm và trải nghiệm thực tế của người dùng.

1.2. Mục tiêu

Từ động cơ trên, đề tài đặt ra các mục tiêu cụ thể sau:

- (1) Xây dựng pipeline thu thập và tiền xử lý dữ liệu giao thông đa nguồn tại TP.HCM, kết hợp dữ liệu động từ TomTom Traffic API, dữ liệu hạ tầng từ OpenStreetMap và Mapbox, dữ liệu thời tiết từ Open-Meteo.
- (2) Nghiên cứu, so sánh và đánh giá năm thuật toán gom cụm (K-Means, Mini-BatchKMeans, DBSCAN, Gaussian Mixture Model, Agglomerative Clustering) trên dữ liệu thực tế của TP.HCM; lựa chọn mô hình tốt nhất theo các chỉ số Silhouette Score, Davies–Bouldin Index, Calinski–Harabasz Index và các chỉ số ngoại bộ ARI, NMI.
- (3) Hiện thực mô hình gom cụm tốt nhất lên ứng dụng di động TraffiGo: tô màu sáu mức trạng thái giao thông trên bản đồ, định tuyến né khu vực kẹt, chấm điểm sức khỏe tuyến đường và ước lượng chi phí nhiên liệu.
- (4) Xây dựng server Machine Learning bằng FastAPI cung cấp bộ REST API dự đoán tốc độ lưu thông, mức độ kẹt xe, rủi ro sự cố giao thông và gợi ý tuyến đường tối ưu; tích hợp server vào ứng dụng di động theo kiến trúc client–server có cơ chế fallback an toàn.
- (5) Kiểm thử toàn diện hệ thống ở cả mức đơn vị lẫn mức chức năng trên thiết bị giả lập trước khi bàn giao và báo cáo.

1.3. Phạm vi

Về dữ liệu, đề tài tập trung vào khu vực Thành phố Hồ Chí Minh với tập snapshot giao thông thu thập trong khoảng 09/03–15/04/2026; dữ liệu được lấy gián tiếp qua các API trung gian (TomTom, OpenStreetMap, Mapbox, Open-Meteo), chưa khai thác trực tiếp camera giám sát hay cảm biến vật lý đặt tại nút giao thông. Về bài toán, đề tài tập trung vào phân tích và nhận diện trạng thái giao thông bằng gom cụm, cùng bài toán dự đoán tốc độ/mức kẹt ở mức theo đoạn đường; các bài toán nâng cao như dự báo chuỗi thời gian dài hạn (LSTM, Graph Neural Network), tối ưu tín hiệu đèn giao thông hay điều phối giao thông tự động nằm ngoài phạm vi hiện tại và được ghi nhận là hướng phát triển. Về sản phẩm, ứng dụng di động nhắm tới người dùng cá nhân trên nền Android; phiên bản web dành cho nhà quản trị và các bài toán triển khai quy mô lớn (đà thành phố, dữ liệu phân tán) chưa thuộc phạm vi của đồ án này.

1.4. Ý nghĩa

1.4.1. Ý nghĩa thực tiễn

Đối với người tham gia giao thông, hệ thống cung cấp một ứng dụng di động tích hợp đầy đủ các nhu cầu khi đi đường: bản đồ trạng thái giao thông trực quan với sáu mức màu

phân biệt theo cụm, định tuyến ưu tiên đoạn thoáng, ước lượng thời gian sát thực tế, ước tính chi phí nhiên liệu, tin tức giao thông tổng hợp từ năm báo điện tử, trợ lý giọng nói, gọi xe nhanh tới các nhà cung cấp phổ biến và bản đồ ngoại tuyến. Kết quả phân cụm giúp người dùng *hiểu* vì sao một tuyến được đề xuất thay vì chỉ nhận kết quả vô cảm. Đối với nhà quản lý và nhà phát triển, bộ API Machine Learning và pipeline dữ liệu mở là nền tảng sẵn sàng cho các ứng dụng giao thông thông minh (ITS) tiếp theo, không bị khóa cứng vào một sản phẩm thương mại nào.

1.4.2. Ý nghĩa khoa học

Về mặt học thuật, đề tài đóng góp ba điểm chính:

- (i) đánh giá có hệ thống năm thuật toán gom cụm trên dữ liệu giao thông thực tế của TP.HCM với bộ 5–10 chỉ số đánh giá nội bộ và ngoại bộ, thay vì dùng ở một chỉ số duy nhất;
- (ii) chứng minh hiệu quả của bước lựa chọn thuộc tính (Feature Selection) trong bài toán gom cụm giao thông — Silhouette Score cải thiện từ 0,1310 lên 0,4183 khi giảm từ 40 xuống 10 thuộc tính;
- (iii) chỉ ra rằng mô hình gom cụm ở dạng JSON (tâm cụm + tham số chuẩn hóa) có thể nhúng trực tiếp lên thiết bị di động để suy luận on-device, kết hợp với mô hình dự đoán giám sát phía server, tạo thành kiến trúc lai phù hợp với điều kiện mạng thực tế.

1.5. Cấu trúc báo cáo

Báo cáo gồm tám chương, được tổ chức theo cấu trúc chuẩn của biểu mẫu báo cáo Đồ án tốt nghiệp của Khoa KH&KT Máy tính. Chương 1 giới thiệu động cơ, mục tiêu, phạm vi và ý nghĩa của đề tài. Chương 2 trình bày kiến thức và công nghệ nền tảng, bao gồm các kỹ thuật gom cụm, chuẩn hóa dữ liệu, lựa chọn thuộc tính, các chỉ số đánh giá và stack công nghệ của cả ứng dụng lẫn server. Chương 3 xem xét các công trình nghiên cứu và hệ thống thực tế liên quan, chỉ ra khoảng trống mà đề tài hướng tới. Chương 4 mô tả hệ thống đề xuất và các yêu cầu chức năng, phi chức năng, dữ liệu. Chương 5 trình bày phân tích nghiệp vụ, mô hình hóa tương tác và thiết kế hệ thống (kiến trúc, cơ sở dữ liệu, chi tiết bên trong, giao diện). Chương 6 mô tả hiện thực và kiểm thử, trình bày kết quả thực nghiệm của các mô hình cùng kết quả tích hợp trên ứng dụng. Chương 7 đánh giá hệ thống theo ưu điểm và hạn chế. Chương 8 tổng kết kết quả đạt được và định hướng phát triển. Cuối báo cáo là kế hoạch thực hiện, tài liệu tham khảo và hai phụ lục kỹ thuật.

CHƯƠNG 2

KIẾN THỨC VÀ CÔNG NGHỆ NỀN TẢNG

2.1. Bài toán nhận diện trạng thái giao thông và hướng tiếp cận không giám sát

Trạng thái giao thông của một đoạn đường được đặc trưng bởi một vector thuộc tính gồm ba nhóm:

- (i) *giao thông động* — tốc độ hiện tại, tốc độ dòng tự do, chỉ số ùn tắc, độ chiếm dụng;
- (ii) *hạ tầng và hình học* — số làn, giới hạn tốc độ, độ cong, độ dốc, số giao lộ, loại đường;
- (iii) *ngữ cảnh* — thời điểm trong ngày, ngày trong tuần, thời tiết.

Nếu có sẵn nhãn “tắc/trung bình/thoáng” cho từng đoạn đường, bài toán trở thành phân loại có giám sát. Trên thực tế, việc gán nhãn thủ công cho hàng nghìn đoạn đường là bất khả thi và thiếu khách quan; hơn nữa, ranh giới giữa các trạng thái giao thông vốn mờ và phụ thuộc từng đô thị. Học máy không giám sát, đặc biệt là gom cụm, cho phép hệ thống *tự khám phá* các trạng thái từ chính dữ liệu: những đoạn đường có hành vi tương tự sẽ được xếp vào cùng một nhóm, và mỗi nhóm sau đó được diễn giải ngược lại thành trạng thái có ý nghĩa thực tiễn (ví dụ “kẹt hoàn toàn”, “chậm và đông”, “thoáng”).

2.2. Gom cụm dữ liệu

Gom cụm là bài toán học máy không giám sát [1]: cho tập dữ liệu $X = \{x_1, x_2, \dots, x_n\} \subset \mathbb{R}^d$, cần phân chia các điểm vào K nhóm sao cho các điểm trong cùng một cụm tương tự nhau và khác biệt với các điểm thuộc cụm khác. Độ tương tự thường được đo bằng khoảng cách Euclid:

$$d(x_i, x_j) = \sqrt{\sum_{l=1}^d (x_{il} - x_{jl})^2}. \quad (2.1)$$

Tùy giả định về hình dạng cụm và bản chất dữ liệu, các thuật toán gom cụm chia thành bốn nhóm chính: phân hoạch, mật độ, mô hình xác suất và phân cấp. Đề tài khảo sát một đại diện tiêu biểu cho mỗi nhóm.

2.2.1. Nhóm phân hoạch: K-Means

K-Means [2, 3] là thuật toán gom cụm phổ biến nhất nhờ chi phí tính toán thấp và khả năng mở rộng tốt. Với số cụm K xác định trước, mục tiêu của K-Means là tối thiểu hóa tổng bình phương khoảng cách từ mỗi điểm đến tâm cụm của nhóm nó thuộc về:

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \rightarrow \min, \quad (2.2)$$

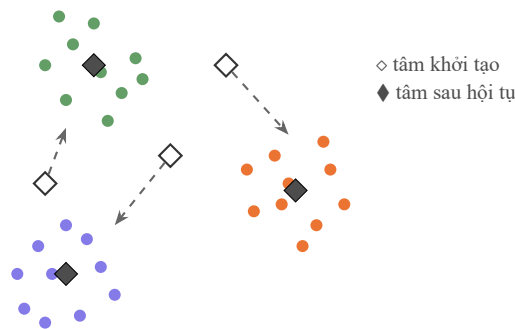
trong đó C_k là cụm thứ k và μ_k là tâm cụm tương ứng. Thuật toán hoạt động theo hai bước lặp cho tới khi hội tụ:

Bước 1. Gán cụm: mỗi điểm x_i được gán vào cụm có tâm gần nhất, $k^* = \arg \min_k \|x_i - \mu_k\|^2$.

Bước 2. Cập nhật tâm: μ_k mới bằng trung bình cộng của toàn bộ điểm thuộc cụm k .

Vì bài toán (2.2) là bài toán NP-khó, K-Means chỉ tìm nghiệm cục bộ phụ thuộc vào khởi tạo tâm ban đầu; trong thực nghiệm, thuật toán k -means++ được dùng để khởi tạo các tâm cách nhau hợp lý, giảm khả năng hội tụ vào nghiệm kém.¹ Độ phức tạp mỗi vòng lặp là $O(nKd)$ — tuyến tính theo số điểm, phù hợp với tập dữ liệu hàng trăm nghìn mẫu của đề tài. Hạn chế chính của K-Means là phải xác định K trước và giả định cụm có dạng hình cầu, các cụm có kích thước gần tương đương.

Hình 2.1 minh họa cơ chế trên bằng một ví dụ hai chiều với ba cụm: từ ba tâm khởi tạo tạo ban đầu (hình thoi rỗng), thuật toán lần lượt gán điểm vào tâm gần nhất rồi cập nhật lại tâm (hình thoi đặc); sau vài vòng lặp, các tâm hội tụ về trọng tâm thực của từng nhóm điểm. Cơ chế đơn giản này chính là nền tảng để gán *trạng thái giao thông* cho từng đoạn đường ở các chương sau.



Hình 2.1: Minh họa hai bước gán cụm và cập nhật tâm của K-Means trên dữ liệu hai chiều.

2.2.2. Nhóm phân hoạch: MiniBatchKMeans

MiniBatchKMeans là biến thể tối ưu tốc độ của K-Means: thay vì dùng toàn bộ dữ liệu, mỗi vòng lặp chỉ sử dụng một lô con ngẫu nhiên (mini-batch) để gán cụm và cập nhật

¹ k -means++ chọn tâm đầu tiên ngẫu nhiên, các tâm tiếp theo được chọn với xác suất tỉ lệ thuận với bình phương khoảng cách tới tâm gần nhất đã chọn — giúp các tâm phủ đều dữ liệu.

tâm theo tốc độ học giảm dần. Cách làm này giảm mạnh thời gian huấn luyện trên tập dữ liệu lớn, tuy nhiên do chỉ cập nhật tâm dựa trên tập con, kết quả có thể kém ổn định hơn K-Means truyền thống nếu kích thước lô quá nhỏ. Với dữ liệu giao thông có hàng trăm nghìn mẫu, MiniBatchKMeans là ứng viên đáng khảo sát khi cân nhắc giữa chất lượng và thời gian chạy.

2.2.3. Nhóm mật độ: DBSCAN

DBSCAN [4] (Density-Based Spatial Clustering of Applications with Noise) phân cụm dựa trên mật độ điểm. Hai tham số điều khiển thuật toán: bán kính lân cận ϵ và số điểm tối thiểu $minPts$. Một điểm được gọi là *điểm lõi* nếu trong bán kính ϵ quanh nó có ít nhất $minPts$ điểm; các điểm lõi kết nối trực tiếp hoặc gián tiếp với nhau tạo thành cụm; những điểm không thuộc cụm nào được đánh dấu là nhiễu (noise). Ưu điểm nổi bật của DBSCAN là không cần xác định trước số cụm, phát hiện được cụm có hình dạng bất kỳ và có khả năng loại bỏ điểm ngoại lệ — một đặc tính hữu ích với dữ liệu giao thông chứa các bản ghi bất thường. Nhược điểm là kết quả nhạy cảm với việc chọn ϵ và khó hoạt động ổn định khi mật độ dữ liệu không đồng đều giữa các vùng.

2.2.4. Nhóm mô hình xác suất: Gaussian Mixture Model

GMM [5] giả định dữ liệu được sinh từ hỗn hợp K phân phối Gaussian nhiều biến:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k), \quad \sum_{k=1}^K \pi_k = 1, \quad (2.3)$$

với π_k , μ_k , Σ_k lần lượt là trọng số, vector trung bình và ma trận hiệp phương sai của thành phần thứ k . Tham số được ước lượng bằng thuật toán EM (Expectation–Maximization): bước E tính xác suất hậu nghiệm γ_{ik} rằng điểm x_i thuộc thành phần k ; bước M cập nhật lại (π_k, μ_k, Σ_k) dựa trên các xác suất này. Khác với K-Means gán cứng mỗi điểm vào đúng một cụm, GMM cho phép gán mềm theo xác suất và hỗ trợ cụm elip nhờ ma trận hiệp phương sai đầy đủ; đổi lại, chi phí tính toán và bộ nhớ cao hơn, và mô hình dễ bị overfitting khi số chiều lớn mà số mẫu không đủ.

2.2.5. Nhóm phân cấp: Agglomerative Clustering

Agglomerative Clustering thuộc nhóm phân cấp bottom-up: khởi đầu mỗi điểm dữ liệu là một cụm, sau đó thuật toán liên tục gộp hai cụm gần nhau nhất theo một tiêu chí liên kết (single-linkage, complete-linkage, average-linkage hoặc Ward) cho tới khi đạt số cụm mong muốn. Phương pháp này không yêu cầu khởi tạo ngẫu nhiên nên kết quả deterministic, đồng thời sinh ra cả cây phân cấp (dendrogram) giúp quan sát cấu trúc dữ liệu ở nhiều

mức chi tiết. Tuy nhiên, độ phức tạp $O(n^2)$ đến $O(n^3)$ về thời gian và $O(n^2)$ về bộ nhớ khiến nó khó mở rộng cho tập dữ liệu rất lớn.

2.2.6. So sánh các thuật toán

Bảng 2.1 tổng hợp đặc tính của năm thuật toán khảo sát trên các tiêu chí then chốt của bài toán giao thông: số cụm xác định trước, hình dạng cụm, độ phức tạp và khả năng xử lý nhiễu.

Bảng 2.1: So sánh đặc tính các thuật toán gom cụm khảo sát.

Thuật toán	Số cụm	Hình dạng cụm	Độ phức tạp	Đặc điểm chính
K-Means	Có (K)	Hình cầu	$O(nKd)$ mỗi vòng	Nhanh, mở rộng tốt, nhạy khởi tạo
MiniBatchKMeans	Có (K)	Hình cầu	$O(bKd)$ mỗi vòng	Nhanh hơn, kém ổn định hơn
DBSCAN	Không	Bất kỳ	$O(n \log n)$ trung bình	Loại nhiễu tốt, nhạy ϵ
GMM	Có (K)	Elip	$O(nKd^2)$ mỗi vòng EM	Gán mềm, tốn bộ nhớ
Agglomerative	Có (K)	Tùy linkage	$O(n^2)$ – $O(n^3)$	Deterministic, khó mở rộng

2.2.7. Đánh giá mức phù hợp với bài toán giao thông

Đối chiếu với đặc điểm tập dữ liệu của đề tài — hàng trăm nghìn mẫu, 10–40 chiều, các cụm kỳ vọng có dạng “tâm tụm” quanh các mức congestionIndex (vì bản chất giao thông là liên tục từ thoáng tới kẹt) — có thể dự đoán trước mức phù hợp của từng nhóm thuật toán. K-Means phù hợp nhất về mặt lý thuyết: các trạng thái giao thông vốn phân tầng theo tốc độ tương đối nên cụm hình cầu với tâm là “mức trạng thái” phản ánh đúng bản chất; chi phí tuyến tính cho phép chạy lại toàn bộ pipeline trong khoảng một phút. MiniBatchKMeans là phương án dự phòng khi dữ liệu phình to, tuy nhiên với 150–170 nghìn mẫu hiện tại, khoảng cách thời gian giữa hai phiên bản còn nhỏ nên giá trị lựa chọn chưa nổi bật. DBSCAN mạnh về loại nhiễu nhưng mật độ đoạn giao thông rất không đồng đều (nội đô dày, ngoại thành thưa) khiến một ϵ duy nhất khó thỏa mọi vùng; kết quả thực nghiệm ở Chương 6 xác nhận điều này (Silhouette âm). GMM có ưu thế gán mềm nhưng tốn kém với dữ liệu one-hot nhiều chiều; Agglomerative deterministic nhưng bậc hai về bộ nhớ không khả thi cho mở rộng quy mô. Các dự đoán lý thuyết này sẽ được kiểm chứng bằng số liệu thực nghiệm.

2.3. Chuẩn hóa dữ liệu và lựa chọn thuộc tính

2.3.1. Chuẩn hóa

Vì khoảng cách Euclid (2.1) phụ thuộc đơn vị đo, các thuộc tính có miền giá trị lớn (ví dụ trafficVolume tới hàng nghìn) sẽ “lấn át” các thuộc tính có miền nhỏ (ví dụ curvatureIndex cỡ phần trăm) nếu không được chuẩn hóa. Đề tài sử dụng chuẩn hóa z-score (StandardScaler): mỗi thuộc tính được biến đổi theo

$$z = \frac{x - \mu}{\sigma}, \quad (2.4)$$

với μ , σ là trung bình và độ lệch chuẩn của thuộc tính trên tập huấn luyện. Sau chuẩn hóa, mọi thuộc tính có trung bình 0 và phương sai 1, đảm bảo các thuật toán dựa trên khoảng cách (K-Means, DBSCAN, GMM) đóng góp công bằng cho mọi chiều dữ liệu.

2.3.2. Lựa chọn thuộc tính

Không phải mọi thuộc tính đều hữu ích: các thuộc tính dư thừa, nhiễu hoặc không liên quan làm loãng không gian đặc trưng, giảm chất lượng gom cụm và tăng thời gian chạy. Đề tài khảo sát hai phương pháp lựa chọn thuộc tính:

- (i) **Mutual Information** [6] — đo lượng thông tin chia sẻ giữa thuộc tính và nhãn tham chiếu, ưu tiên thuộc tính có MI cao;
- (ii) **Random Forest Importance** [7] — dựa trên mức độ giảm tạp chất (impurity decrease) trung bình khi thuộc tính được dùng để chia nút trong rừng cây quyết định.

Cách tiếp cận top- k được áp dụng: sắp xếp thuộc tính theo điểm số rồi thử nghiệm với các giá trị k khác nhau, chọn tập cho chất lượng gom cụm tốt nhất (trình bày chi tiết ở Chương 6).

2.4. Các chỉ số đánh giá chất lượng gom cụm

2.4.1. Chỉ số nội bộ (internal)

Silhouette Score [8]: với mỗi điểm x_i trong cụm C_i , đặt $a(i)$ là khoảng cách trung bình tới các điểm cùng cụm và $b(i)$ là khoảng cách trung bình nhỏ nhất tới các điểm của cụm lân cận gần nhất; điểm số silhouette của x_i là

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1]. \quad (2.5)$$

Giá trị gần 1 nghĩa là điểm nằm gọn trong cụm của mình và xa các cụm khác; giá trị gần 0 nghĩa là điểm nằm trên biên hai cụm; giá trị âm nghĩa là điểm bị gán nhầm cụm. Chỉ số tổng thể là trung bình $s(i)$ trên toàn bộ dữ liệu, càng cao càng tốt.

Davies–Bouldin Index [9]: đo trung bình “tỉ lệ phân tán trên khoảng cách” giữa mỗi cặp cụm,

$$DBI = \frac{1}{K} \sum_{k=1}^K \max_{j \neq k} \frac{S_k + S_j}{M_{kj}}, \quad (2.6)$$

với S_k là bán kính hiệu dụng (độ phân tán trung bình) của cụm k và M_{kj} là khoảng cách giữa hai tâm cụm. DBI càng *thấp* chứng tỏ các cụm càng tách biệt và gọn gàng.

Calinski–Harabasz Index [10]: tỉ lệ giữa phương sai giữa-cụm và phương sai trong-cụm, được chuẩn hóa bởi bậc tự do:

$$CH = \frac{\text{tr}(B_K)}{\text{tr}(W_K)} \cdot \frac{n - K}{K - 1}, \quad (2.7)$$

trong đó B_K là ma trận phân tán giữa-cụm và W_K là ma trận phân tán trong-cụm. CH càng *cao* càng tốt.

2.4.2. Chỉ số ngoại bộ (external)

Khi có nhãn trạng thái giao thông tham chiếu (do đề tài xây dựng từ ngưỡng LOS và tốc độ tương đối, trình bày ở Chương 6), chất lượng gom cụm có thể đánh giá bằng mức khớp với nhãn tham chiếu: **ARI** (Adjusted Rand Index) [11] đo mức đồng thuận của mọi cặp điểm giữa hai phân hoạch, hiệu chỉnh cho trường hợp trùng khớp ngẫu nhiên, $\in [-1, 1]$; **NMI** (Normalized Mutual Information) đo lượng thông tin chia sẻ giữa hai phân hoạch đã chuẩn hóa về $[0, 1]$; **FMI** (Fowlkes–Mallows) và **F-Measure** là đại lượng hình học trung bình của độ chính xác và độ phủ cặp điểm; **Entropy** đo mức hỗn tạp nhãn tham chiếu trong từng cụm (càng thấp càng tốt).

2.5. Nguồn dữ liệu giao thông và thời tiết

Đề tài tổng hợp dữ liệu từ bốn nguồn, mỗi nguồn đảm nhận một nhóm thuộc tính và được thu thập định kỳ thành các snapshot. Bảng 2.2 so sánh vai trò và đặc điểm của từng nguồn.

TomTom Flow Segment Data API trả về trạng thái lưu thông tại một điểm theo hai chỉ số chính: *currentSpeed* (tốc độ hiện tại) và *freeFlowSpeed* (tốc độ khi dòng xe thông thoáng hoàn toàn). Từ hai giá trị này, đề tài định nghĩa chỉ số *congestionIndex* = $\text{currentSpeed}/\text{freeFlowSpeed} \in [0, 1]$; lưu ý theo quy ước của đề tài, *congestionIndex* **thấp** tương ứng trạng thái **kẹt** và **cao** tương ứng **thoáng**.

Bảng 2.2: Các nguồn dữ liệu của đề tài.

Nguồn	Nhóm dữ liệu	Nội dung khai thác
TomTom Traffic [12]	Giao thông động	currentSpeed, freeFlowSpeed, jamFactor, độ tin cậy theo đoạn đường
OpenStreetMap [13]	Hạ tầng, hình học	Hình học tuyến, loại đường, số làn, hướng một chiều, giới hạn tốc độ
Mapbox	Định hướng, làn xe	Directions, lane count, map-matching confidence
Open-Meteo [14]	Thời tiết	Nhiệt độ, lượng mưa, tốc độ gió theo ngày và tọa độ

2.6. Công nghệ phát triển ứng dụng di động

Ứng dụng TraffiGo được phát triển bằng **Java** trên nền Android SDK (minSdk 29, targetSdk 36), kiến trúc theo Activity — mỗi màn hình là một Activity, logic nghiệp vụ đặt trực tiếp trong activity, sử dụng ViewBinding để truy cập view an toàn kiểu dữ liệu. Các thành phần chính:

- **Google Maps SDK for Android:** hiển thị bản đồ, vẽ polyline theo màu cụm, marker, camera animation; yêu cầu OpenGL ES 2.0.
- **Google Routes API:** tính tuyến đường cho phương tiện ô tô/xe máy/đi bộ, hỗ trợ điểm dừng trung gian (intermediates) và trả về polyline mã hóa cần giải mã (encoded polyline).
- **Google Places API:** gợi ý và tìm địa điểm theo từ khóa với location bias.
- **Firebase Authentication:** đăng nhập email/mật khẩu và Google Sign-In.
- **Firebase Realtime Database:** lưu hồ sơ người dùng, địa điểm đã lưu, lịch sử lộ trình, avatar Base64 và sự cố cộng đồng; đồng bộ thời gian thực.
- **OkHttp:** thư viện HTTP client bất đồng bộ dùng cho mọi lời gọi mạng, kể cả tới server ML.
- **osmdroid + bản đồ chụp màn hình:** phục vụ chế độ bản đồ ngoại tuyến khi không có kết nối.

2.7. Công nghệ xây dựng server Machine Learning

Server ML được viết bằng **Python 3** với các thành phần:

- **scikit-learn** [16]: huấn luyện mô hình (Random Forest Regressor, StandardScaler, K-Means, one-hot encoding) và các hàm đánh giá.
- **joblib:** serialize mô hình đã huấn luyện ra tệp .joblib để nạp lại khi khởi chạy server.
- **FastAPI:** framework REST API hiện đại dựa trên kiểu chú thích Pydantic — tự xác thực dữ liệu vào/ra, tự sinh tài liệu Swagger UI, hỗ trợ xử lý bất đồng bộ.
- **Uvicorn:** ASGI server chạy ứng dụng FastAPI, hỗ trợ nhiều worker cho tải đồng

thời.

Một điểm thiết kế quan trọng: mô hình gom cụm K-Means không được serialize qua joblib mà được *xuất ra JSON* (danh sách thuộc tính, tham số scaler, ma trận tâm cụm, thứ hạng nghiêm trọng). Cách làm này giúp **cùng một tệp mô hình chạy được ở cả ba môi trường**: pipeline Python, server Python và ứng dụng Android (nơi không thể cài scikit-learn), đảm bảo nhất quán tuyệt đối giữa kết quả phân tích và hiển thị.

Bảng 2.3 so sánh ba phương án framework back-end đã cân nhắc trên các tiêu chí then chốt của đề tài; FastAPI vượt trội ở khả năng tự sinh tài liệu và tích hợp mô hình cùng tiến trình — hai yếu tố quyết định cho bản chất “demo sẵn sàng” của server ML.

Bảng 2.3: So sánh các phương án framework cho server ML.

Tiêu chí	FastAPI (chọn)	Flask	Node.js/Express
Xác thực schema vào/ra	Tự động (Pydantic)	Thủ công	Thủ công (middleware)
Tài liệu API	Tự sinh Swagger UI	Cần extension	Cần công cụ ngoài
Chạy mô hình sklearn	Cùng tiến trình	Cùng tiến trình	Cần service Python riêng
Xử lý bất đồng bộ	Async/await gốc	Đồng bộ (WSGI)	Event-loop gốc
Hiệu năng (bài toán nhỏ)	Tốt	Đủ dùng	Tốt
Độ phức tạp triển khai	Thấp	Thấp nhất	Trung bình (2 dịch vụ)

2.8. Kiến trúc ứng dụng Android và mô hình thực thi

Ứng dụng TraffiGo tổ chức theo mô hình Activity truyền thống (không dùng Fragment/ViewModel): mỗi màn hình là một Activity độc lập, dữ liệu trao đổi qua Intent, logic nghiệp vụ đặt trực tiếp trong activity. Lựa chọn này đánh đổi tính “hiện đại” của kiến trúc MVVM lấy lại ba lợi thế thực dụng:

- (i) luồng điều hướng đơn giản dễ demo và dễ gỡ lỗi;
- (ii) không phải học thêm bộ state management;
- (iii) toàn bộ vòng đời tài nguyên (bản đồ, TTS, recognizer, listener Firebase) nằm gọn trong onCreate—onDestroy của từng activity, dễ kiểm soát rõ ràng.

Ba mô hình thực thi song song quan trọng cần nắm khi đọc mã nguồn:

- (1) **Luồng chính (main thread)** xử lý mọi thao tác UI và callback bản đồ; Google Maps SDK cấm gọi API bản đồ từ luồng khác (lỗi “Not on the main thread”), nên mọi truy vấn camera/projection phải được thực hiện ở main thread rồi mới chuyển dữ liệu xuống luồng nền;

- (2) **Luồng nền (executor)** chạy các tác vụ nặng: nạp 23.417 đoạn hình học, phân tích spatial grid, chụp và nén ảnh bản đồ ngoại tuyến; kết quả được công bố về UI qua `runOnUiThread` kèm cờ `isActive` chặn callback muộn sau khi activity đã hủy;
- (3) **Luồng bất đồng bộ thư viện** (OkHttp callbacks, Firebase listeners) vốn mặc định trả kết quả trên main thread.

Về quyền truy cập, ứng dụng yêu cầu `ACCESS_FINE_LOCATION` (định vị), `INTERNET` và `RECORD_AUDIO` (trợ lý giọng nói); quyền vị trí được xin runtime theo hai kịch bản (xem bản đồ, dẫn đường) và sau khi cấp phải tiếp tục đúng hành động người dùng đang thực hiện.

2.9. Kết chương

Chương này đã trình bày nền tảng lý thuyết và công nghệ của đề tài: bốn nhóm thuật toán gom cụm với các công thức cốt lõi và đặc tính riêng, chuẩn hóa z-score và hai phương pháp lựa chọn thuộc tính, bộ chỉ số đánh giá nội bộ lẫn ngoại bộ, bốn nguồn dữ liệu giao thông—thời tiết, cùng stack công nghệ của ứng dụng Android và server FastAPI/scikit-learn. Những kiến thức này là cơ sở trực tiếp cho phần phân tích, thiết kế (Chương 5) và hiện thực, kiểm thử (Chương 6).

CHƯƠNG 3

CÔNG TRÌNH LIÊN QUAN

3.1. Tổng quan các hướng tiếp cận

Các nghiên cứu về phân tích và dự đoán giao thông bằng học máy đã phát triển theo nhiều hướng: dự đoán dòng xe chuỗi thời gian (LSTM, Graph Neural Network), phân loại trạng thái bằng mô hình có giám sát, và hướng *gom cụm* — nhận diện các mẫu hình giao thông điển hình mà không cần nhãn. Trong đó, hướng gom cụm phù hợp với bối cảnh Việt Nam nơi dữ liệu gán nhãn khan hiếm, đồng thời kết quả gom cụm có tính diễn giải cao, dễ chuyển hóa thành sản phẩm trực quan. Ba công trình tiêu biểu dưới đây được khảo sát chi tiết.

3.2. Ba công trình nghiên cứu tiêu biểu

Công trình 1 — A clustering-based traffic flow prediction method with dynamic spatio-temporal correlation analysis dự báo dòng xe bằng cách gom cụm các đoạn đường theo mẫu hình chuỗi thời gian thành các “nhóm có tương quan”, rồi dự báo theo nhóm với cơ chế cập nhật tương quan động. Độ chính xác cao hơn phương pháp dự báo từng đoạn độc lập. *Bài học cho đề tài*: gom cụm theo *đoạn đường* (mỗi đoạn là một điểm dữ liệu) phù hợp với dữ liệu snapshot đang có.

Công trình 2 — Clustering of Urban Traffic Patterns by K-Means and Dynamic Time Warping phân loại mẫu hình giao thông đô thị bằng K-Means trên độ đo DTW, cho ra các cụm phản ánh rõ kiểu ngày giao thông với tính diễn giải tốt. *Bài học cho đề tài*: độ đo tương tự quan trọng không kém thuật toán; với dữ liệu snapshot rời rạc, khoảng cách Euclid trên vector đặc trưng phù hợp hơn DTW.

Công trình 3 — A Spatiotemporal Clustering-Based Route Recommendation System for Travel Time Estimation gom cụm “trạng thái mạng lưới” theo không gian–thời gian để ước lượng thời gian và gợi ý tuyến đường, hoạt động ở mức demo. *Bài học cho đề tài*: kết quả gom cụm chỉ tạo giá trị thực khi được “đóng gói” thành dịch vụ cho người dùng cuối — đây chính là động lực để đề tài phát triển ứng dụng di động và server API thay vì dừng ở dashboard phân tích.

3.2.1. Điểm khác biệt giữa các công trình và đề tài

Bảng 3.1 chốt lại khoảng cách giữa ba công trình khảo sát và giải pháp của đề tài theo năm khía cạnh thực thi. Điểm khác biệt căn bản nhất: các công trình đều dừng ở mức *mô hình* và *đánh giá*, trong khi đề tài kéo kết quả nghiên cứu đến tận *sản phẩm người dùng cuối*

và *dịch vụ API* — hai lớp đòi hỏi các quyết định kỹ thuật hoàn toàn khác (nhúng mô hình lên thiết bị, timeout, fallback, thiết kế giao diện) mà tài liệu học thuật hiếm khi đề cập.

Bảng 3.1: Đối chiếu phạm vi thực thi giữa các công trình và đề tài.

Khía cạnh	Công trình 1–3	Đề tài
Dữ liệu	Chuỗi lưu lượng tổng hợp/thiết lập công khai	Snapshot đa nguồn thực tế TP.HCM (TomTom + OSM + Mapbox + thời tiết)
Mô hình	Gom cụm chuỗi thời gian (DTW, tương quan động)	Gom cụm vector đặc trưng đa chỉ số + RF dự đoán
Đánh giá	1–2 chỉ số chính	5–10 chỉ số nội bộ + ngoại bộ
Ứng dụng	Dashboard phân tích (nếu có)	Ứng dụng di động hoàn chỉnh cho người đi đường
Triển khai	Không có API	REST API ML + suy luận on-device + fallback

3.3. Các hệ thống và công cụ thực tế liên quan

Google Maps / Waze: cung cấp điều hướng và trạng thái kẹt xe theo thời gian thực dựa trên đội xe người dùng; trạng thái hiển thị theo ngưỡng màu cố định (đỏ/cam/xanh), không phân nhóm theo đặc thù từng đô thị và không mở API phân tích tùy biến. **TomTom Go:** tương tự, jamFactor được tính từ dữ liệu đội xe và hiển thị theo thang cố định. **INRIX, HERE Traffic Analytics:** các nền tảng phân tích giao thông thương mại hướng tới doanh nghiệp và cơ quan quản lý, chi phí bản quyền cao, không phù hợp người dùng cá nhân tại Việt Nam. **Grab, Be, Xanh SM:** các ứng dụng gọi xe phổ biến tại Việt Nam tập trung vào luồng đặt xe, chỉ hiển thị bản đồ ở mức cơ bản, không cung cấp phân tích trạng thái giao thông theo cụm cho người dùng.

3.4. Nhận xét chung và khoảng trống nghiên cứu

Bảng 3.2 đối chiếu các giải pháp đã khảo sát trên các tiêu chí then chốt của đề tài.

Bảng 3.2: Đối chiếu các giải pháp liên quan với đề tài.

Giải pháp	Gom cụm	Đa nguồn	App người dùng	API ML	Dữ liệu TP.HCM
Công trình 1	✓	—	—	—	—
Công trình 2	✓	—	—	—	—
Công trình 3	✓	—	✓ (demo)	—	—
Google Maps / Waze	—	✓	✓	—	✓ (ẩn)
INRIX / HERE	✓	✓	✓	✓	—
Grab / Be / Xanh SM	—	✓	✓	—	✓ (mức cơ bản)
Đề tài	✓	✓	✓	✓	✓

Từ bảng đối chiếu, khoảng trống nghiên cứu được nhận diện rõ: các công trình học thuật dừng ở mức mô hình và dashboard, các sản phẩm thương mại đóng dữ liệu và mô hình; chưa có giải pháp nào kết hợp *cả ba* yếu tố:

- (i) phân tích gom cụm có đánh giá đa chỉ số trên dữ liệu đa nguồn thực tế của TP.HCM;
- (ii) ứng dụng di động đầy đủ tính năng cho người dùng cá nhân;
- (iii) server ML mở với bộ API dự đoán theo đoạn đường.

Đây chính là vị trí mà đề tài đặt mình vào.

3.5. Kết chương

Chương này đã khảo sát ba công trình nghiên cứu tiêu biểu về gom cụm giao thông cùng các hệ thống thực tế, rút ra các bài học về lựa chọn độ đo, cấu trúc dữ liệu và nhu cầu “đóng gói” kết quả thành sản phẩm; đồng thời chỉ ra khoảng trống mà đề tài lấp đầy. Các kiến thức này trực tiếp định hướng thiết kế hệ thống ở chương tiếp theo.

CHƯƠNG 4

HỆ THỐNG ĐỀ XUẤT

4.1. Mô tả hệ thống

Hệ thống đề xuất của đề tài gồm bốn thành phần liên kết với nhau theo luồng dữ liệu “từ nguồn tới người dùng”:

- (1) **Pipeline dữ liệu:** thu thập định kỳ snapshot giao thông từ TomTom Flow API cho các đoạn đường do OpenStreetMap định nghĩa, kết hợp đặc trưng hạ tầng (OSM/Mapbox) và thời tiết (Open-Meteo), làm sạch và tổng hợp thành tập dữ liệu chuẩn theo từng đoạn — thời điểm.
- (2) **Pipeline học máy:** tiền xử lý, chuẩn hóa, lựa chọn thuộc tính, gom cụm và đánh giá đa chỉ số; mô hình tốt nhất (K-Means $k = 6$) được xuất sang JSON để nhúng lên di động; đồng thời huấn luyện mô hình Random Forest dự đoán tốc độ lưu thông phục vụ server ML.
- (3) **Ứng dụng di động TraffiGo (Android):** bản đồ giao thông sáu mức màu theo cụm, chỉ đường né kẹt với chấm điểm sức khỏe tuyến, tin tức giao thông, địa điểm lân cận, gọi xe nhanh, trợ lý giọng nói và bản đồ ngoại tuyến.
- (4) **Server Machine Learning (FastAPI):** cung cấp bộ REST API dự đoán tốc độ, mức kẹt, rủi ro sự cố, gợi ý và so sánh tuyến đường, tổng hợp trạng thái giao thông; tích hợp hai chiều với ứng dụng theo kiến trúc client–server có fallback.

Hệ thống phục vụ hai nhóm người dùng chính: *người tham gia giao thông cá nhân* — người dùng cuối của ứng dụng TraffiGo; và *nhà phát triển/quản trị* — người vận hành pipeline, huấn luyện lại mô hình và tiêu thụ bộ API ML.

Luồng xử lý dữ liệu tổng quát của hệ thống được chuẩn hóa thành sáu bước ETL lặp lại được, mỗi bước có đầu vào — đầu ra xác định và có thể chạy độc lập:

- Bước 1. Thu thập:** chụp snapshot TomTom Flow cho toàn bộ đoạn trong `geometry.csv` tại khung giờ định sẵn, đặt tên tệp theo mốc thời gian để truy vết.
- Bước 2. Tổng hợp hạ tầng:** nối đặc trưng OSM (loại đường, làn, một chiều, giới hạn tốc độ) và Mapbox (làn, map-matching confidence) vào từng dòng giao thông.
- Bước 3. Làm sạch:** khử trùng lặp theo `segmentId`, chuẩn hóa kiểu số, giới hạn miền giá trị hợp lý, xử lý giá trị thiếu theo từng nhóm cột.

Bước 4. Làm giàu ngữ cảnh: suy hourOfDay, dayOfWeek, dayType từ timeStamp; join thời tiết ngày từ Open-Meteo.

Bước 5. Chuẩn hóa và lựa chọn thuộc tính: z-score; xếp hạng thuộc tính bằng MI và RF Importance; chốt tập top- k .

Bước 6. Xuất bản: đóng gói tập huấn luyện CSV + cluster_model.json + hình ảnh phân tích phục vụ báo cáo.

4.2. Yêu cầu

4.2.1. Yêu cầu chức năng

Bảng 4.1 liệt kê các yêu cầu chức năng (FR) của hệ thống, được đánh giá lại ở chương kiểm thử (Chương 6).

Bảng 4.1: Yêu cầu chức năng của hệ thống.

Mã	Nhóm	Nội dung yêu cầu	Ưu tiên
FR01	Tài khoản	Đăng nhập email/mật khẩu và Google Sign-In bằng Fire-base Auth	Cao
FR02	Bản đồ	Hiển thị trạng thái giao thông thời gian thực trên bản đồ theo 6 cụm màu	Cao
FR03	Bản đồ	Đoạn không có dữ liệu live được tô xám “thiếu dữ liệu”, không suy đoán	Cao
FR04	Bản đồ	Bấm vào đoạn đường hiện chi tiết: tốc độ live, trạng thái, dự báo ML	Cao
FR05	Chỉ đường	Tìm đường ô tô/xe máy/đi bộ, hỗ trợ đa điểm dừng	Cao
FR06	Chỉ đường	Né khu vực kẹt: trọng số thời gian nhân hệ số phạt theo congestionIndex	Cao
FR07	Chỉ đường	Chấm điểm sức khỏe tuyến A–F, cảnh báo tắc, ước tính nhiên liệu	Trung bình
FR08	ML	Gọi server dự báo tốc độ/mức kẹt/rủi ro sự cố cho từng đoạn	Cao
FR09	ML	Nhận đề xuất tuyến từ server và tự chuyển khi server chọn tuyến khác	Trung bình
FR10	ML	Fallback về logic on-device khi server offline, không gián đoạn ứng dụng	Cao
FR11	Tin tức	Tổng hợp tin giao thông từ 5 báo điện tử, lọc danh mục, nghe bài viết	Trung bình
FR12	Địa điểm	Tìm địa điểm lân cận theo danh mục; lưu nhà/công ty/yêu thích/đỗ xe	Trung bình
FR13	Tiện ích	Gọi xe nhanh Grab/Be/Xanh SM; trợ lý giọng nói; bản đồ ngoại tuyến	Trung bình
FR14	Đa ngôn ngữ	Chuyển đổi tiếng Việt/tiếng Anh áp dụng toàn ứng dụng	Thấp

4.2.2. Yêu cầu phi chức năng

Bảng 4.2: Yêu cầu phi chức năng của hệ thống.

Mã	Nhóm	Nội dung yêu cầu
NFR01	Hiệu năng	Suy luận gom cụm on-device; server ML phản hồi ≤ 1 giây mỗi request
NFR02	Hiệu năng	Nạp 23.417 đoạn hình học không khung UI (xử lý nền + spatial grid)
NFR03	Khả dụng	App hoạt động đầy đủ khi không có server ML và giảm chức năng khi offline
NFR04	Tiết kiệm	Giới hạn gọi TomTom theo khung nhìn (zoom ≥ 14 , tối đa 150 đoạn/lần) để kéo dài quota
NFR05	Bảo mật	Khóa API nhạy cảm lưu ngoài git (local.properties); Realtime Database yêu cầu xác thực
NFR06	Bảo trì	Mô hình chuẩn JSON dùng chung pipeline/server/app; tài liệu API tự sinh (Swagger UI)
NFR07	UX	Giao diện thống nhất một hệ thiết kế (màu chủ đạo, bo góc, nút pill) trên mọi màn hình

4.2.3. Yêu cầu dữ liệu

Tập dữ liệu huấn luyện gồm 171 snapshot giao thông thu thập trong khoảng 09/03–15/04/2026, tổng cộng khoảng 170.000 dòng trên 917 đoạn đường sau khi trùng lặp; mỗi dòng là một đoạn — thời điểm với 40 thuộc tính chia thành năm nhóm:

- **Định danh và không gian:** segmentId, name_vn, tọa độ đầu–cuối đoạn.
- **Hạ tầng và hình học:** laneCount_aggregated, speedLimit, roadType, surface, oneway, curvatureIndex, lengthKm, intersectionCount, routeSlopePercent, startElevation/endElevation.
- **Giao thông động:** currentSpeed, freeFlowSpeed, jamFactor, incidentFlag, congestionIndex, trafficVolume, occupancy, crossTime, speedLimitRatio, relativeCongestionIndex.
- **Thời gian và bối cảnh:** timeStamp (định dạng yyMMddHHmm), dayOfWeek, hourOfDay, dayType (Weekday/Weekend).
- **Điều hướng và độ tin cậy:** route_distance_m, route_duration_sec, route_weight, map_match_confidence.

Dữ liệu thời tiết ngày (nhiệt độ cao nhất, lượng mưa, tốc độ gió) cho đúng khoảng thời gian trên được lấy từ Open-Meteo Archive và join theo ngày. Với dữ liệu thời gian thực, hệ thống tiêu thụ TomTom Flow API theo tọa độ, vị trí GPS của người dùng và sự cố cộng đồng lưu trên Firebase Realtime Database.

4.3. Kết chương

Chương này mô tả bốn thành phần của hệ thống, chốt danh mục 14 yêu cầu chức năng, 7 yêu cầu phi chức năng và cấu trúc tập dữ liệu 40 thuộc tính. Các yêu cầu này là đầu vào trực tiếp cho phân tích và thiết kế ở chương sau, đồng thời là căn cứ kiểm nghiệm ở chương kiểm thử.

CHƯƠNG 5

PHÂN TÍCH VÀ THIẾT KẾ

5.1. Phân tích

5.1.1. Mô hình hóa quy trình nghiệp vụ

Nghệ vụ của đề tài được tổ chức thành ba luồng chính chạy song song và hỗ trợ lẫn nhau:

Luồng dữ liệu và mô hình (offline): bắt đầu từ việc thu thập snapshot giao thông định kỳ từ TomTom cho tập đoạn do OpenStreetMap định nghĩa; dữ liệu thô sau đó được làm sạch (khử trùng lặp, xử lý giá trị thiếu), kết hợp đặc trưng hạ tầng và thời tiết, chuẩn hóa, lựa chọn thuộc tính, gom cụm và đánh giá đa chỉ số. Mô hình tốt nhất được xuất ra JSON cho cả ứng dụng di động lẫn server. Hình 5.1 mô tả chi tiết quy trình này dưới dạng sơ đồ khối.

Luồng người dùng (online, trên ứng dụng): người dùng đăng nhập, xem bản đồ trạng thái giao thông được tô màu theo cụm, thực hiện tìm đường — lúc này ứng dụng kết hợp dữ liệu live TomTom, suy luận gom cụm on-device và (khi có) dự báo từ server để chấm điểm và đề xuất tuyến; người dùng có thể phản hồi bằng cách báo sự cố, lưu lịch sử hoặc đánh giá tuyến.

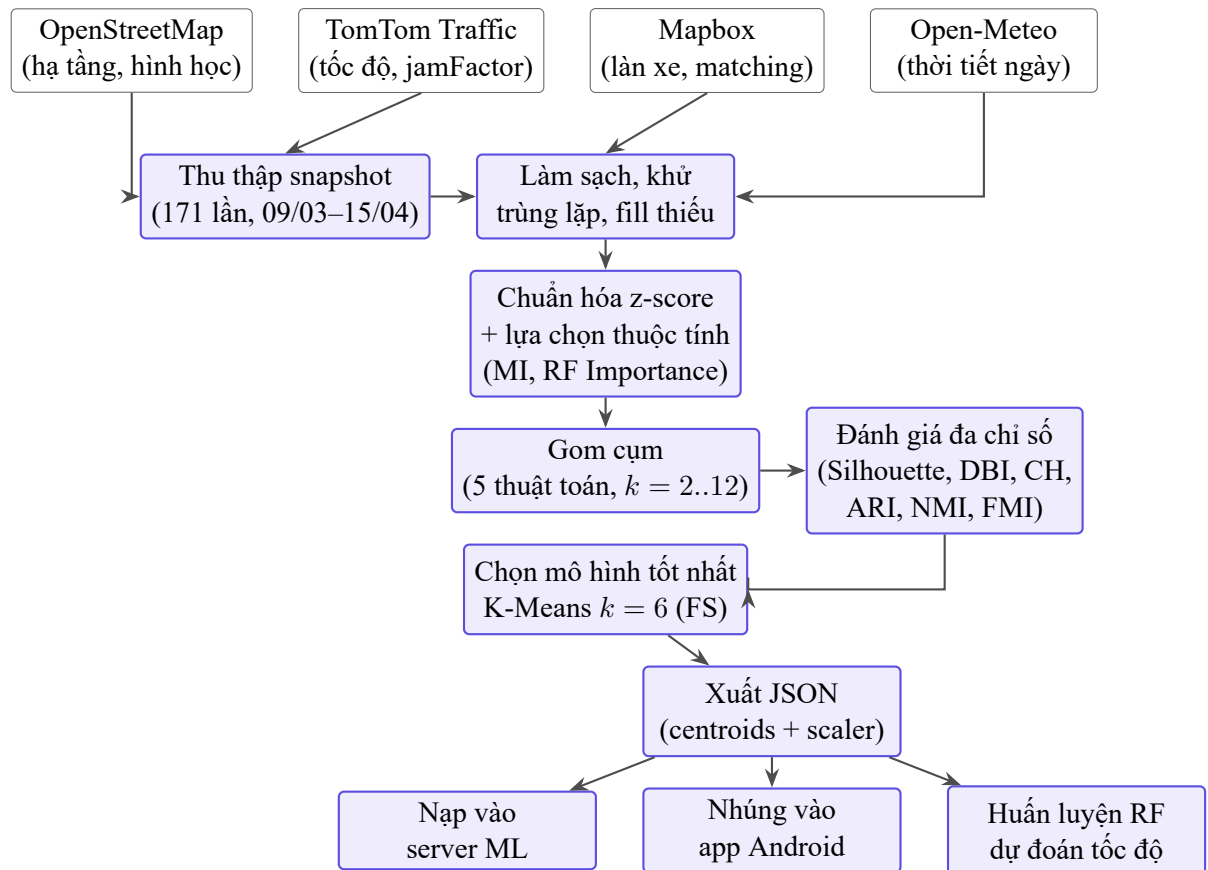
Luồng dịch vụ ML (online, server): ứng dụng gửi đặc trưng của đoạn/tuyến tới server; server chuẩn hóa, suy luận bằng mô hình Random Forest, gán cụm bằng mô hình K-Means dùng chung, tính rủi ro sự cố và trả kết quả; khi không thể kết nối, ứng dụng tự fallback về logic on-device.

5.1.2. Mô hình hóa tương tác

Các tác nhân

Hệ thống có ba tác nhân:

- (i) **Người dùng** — người tham gia giao thông sử dụng ứng dụng di động;
- (ii) **Quản trị** — người vận hành pipeline dữ liệu, huấn luyện lại mô hình và giám sát server ML;
- (iii) **Hệ thống ngoài** — các dịch vụ cung cấp dữ liệu và chức năng (TomTom, Google Maps/Routes/Places, Open-Meteo, Firebase, Gemini) tham gia như các tác nhân phụ trợ trong các ca sử dụng.



Hình 5.1: Sơ đồ khối quy trình nghiệp vụ của pipeline dữ liệu và học máy.

Sơ đồ Use Case

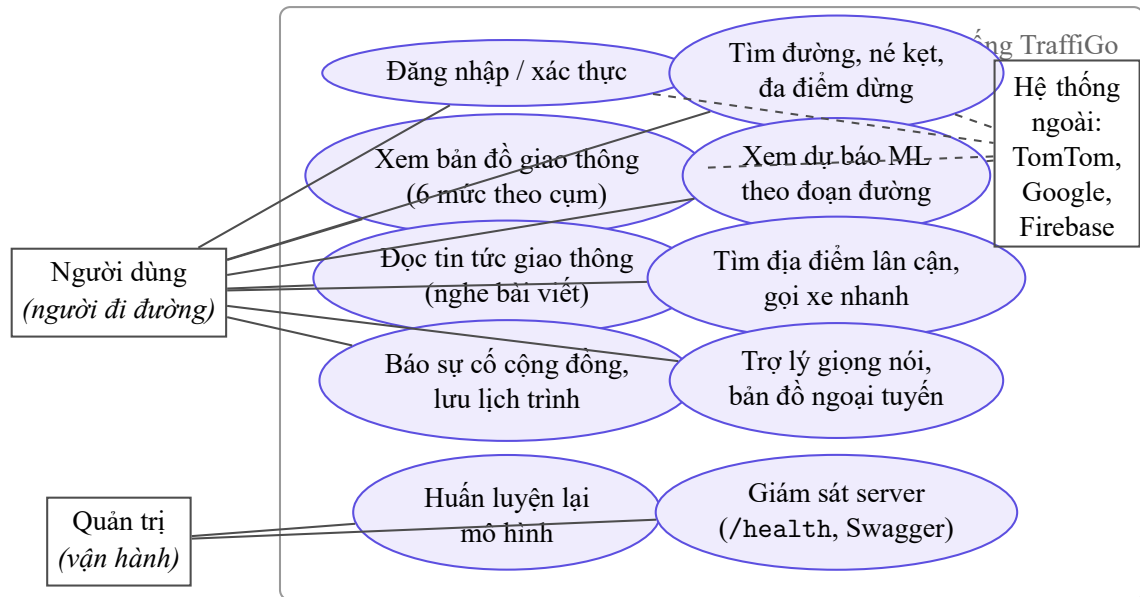
Hình 5.2 trình bày sơ đồ ca sử dụng tổng thể của hệ thống. Người dùng là tác nhân trung tâm với tám nhóm ca sử dụng chính; Quản trị đảm nhận hai ca sử dụng thuộc phần hậu cần; các hệ thống ngoài tham gia vào hầu hết các ca sử dụng online.

Đặc tả các ca sử dụng chính

Sáu ca sử dụng quan trọng nhất được đặc tả chi tiết trong các Bảng 5.1–5.5.

5.1.3. Sơ đồ hoạt động và tuần tự của luồng tìm đường

Hai sơ đồ dưới đây làm rõ luồng nghiệp vụ phức tạp nhất của hệ thống — tìm đường kết hợp dự báo ML. Hình 5.3 mô tả luồng hoạt động với hai điểm rẽ nhánh: kiểm tra khả dụng của server ML và quyết định chuyển tuyến; nhánh fallback đảm bảo luồng không bao giờ bị gián đoạn. Hình 5.4 mô tả tương tác tuần tự giữa bốn thực thể với mười thông điệp, từ khi người dùng xác nhận điểm đến tới khi kết quả hiển thị.



Hình 5.2: Sơ đồ Use Case tổng thể của hệ thống.

Bảng 5.1: Đặc tả UC-01: Đăng nhập và xác thực.

Ca sử dụng	Đăng nhập và xác thực (UC-01)
Tác nhân	Người dùng; hệ thống ngoài: Firebase Authentication
Tiền điều kiện	Ứng dụng đã cài đặt, có kết nối mạng
Luồng chính	1. Người dùng nhập email/mật khẩu hoặc chọn Google Sign-In. 2. Firebase Auth xác thực và trả về thông tin người dùng. 3. Ứng dụng chuyển tới Trang chủ, tải hồ sơ (tên, avatar Base64) và các địa điểm đã lưu từ Realtime Database.
Luồng thay thế	2a. Sai thông tin đăng nhập: hiển thị thông báo lỗi từ Firebase, giữ nguyên màn hình. 2b. Chưa có tài khoản: chuyển sang luồng đăng ký (email/mật khẩu).
Kết quả	Phiên đăng nhập được duy trì (tự khôi phục ở lần mở sau); dữ liệu cá nhân sẵn sàng.
Yêu cầu liên quan	FR01, FR12, NFR05

5.2. Giải pháp công nghệ

5.2.1. Front-end (ứng dụng di động)

Ứng dụng được chọn phát triển **native trên Android bằng Java** thay vì các framework đa nền tảng. Lý do lựa chọn:

- (i) bản đồ là trung tâm của trải nghiệm — Google Maps SDK native cho phép can thiệp sâu vào polyline, camera và gesture mà các wrapper đa nền tảng luôn có khoảng cách một lớp;
- (ii) suy luận gom cụm cần chạy ổn định trên luồng nền với bộ nhớ nhỏ, native Java kiểm soát tốt chi tiết này;

Bảng 5.2: Đặc tả UC-02: Xem bản đồ giao thông.

Ca sử dụng	Xem bản đồ giao thông (UC-02)
Tác nhân	Người dùng
Tiền điều kiện	Ứng dụng đã khởi động, người dùng đã đăng nhập
Luồng chính	1. Ứng dụng nạp hình học 23.417 đoạn từ geometry.csv trên luồng nền. 2. Người dùng di chuyển bản đồ; hệ thống xác định các đoạn trong khung nhìn bằng spatial grid. 3. Với mỗi đoạn hiển thị ($\text{zoom} \geq 14$, tối đa 150 đoạn/lần), gọi TomTom Flow lấy tốc độ live. 4. Tổng hợp 9 đặc trưng và suy luận gom cụm on-device; tô màu theo thứ hạng nghiêm trọng của cụm; đoạn thiếu dữ liệu tô xám. 5. Bấm vào một đoạn: hiện bottom sheet chi tiết kèm thẻ Dự báo ML từ server (nếu khả dụng).
Luồng thay thế	3a. TomTom không có dữ liệu cho đoạn: tô xám, không suy đoán. 4a. Chưa nạp xong mô hình: dùng màu mặc định rồi cập nhật khi model sẵn sàng.
Kết quả	Bản đồ hiển thị trạng thái giao thông 6 mức màu theo thời gian thực.
Yêu cầu liên quan	FR02, FR03, FR04, NFR02, NFR04

Bảng 5.3: Đặc tả UC-03: Tìm đường và né kẹt.

Ca sử dụng	Tìm đường, né kẹt, đa điểm dừng (UC-03)
Tác nhân	Người dùng; hệ thống ngoài: Google Routes API, TomTom, server ML
Tiền điều kiện	Người dùng đã xác nhận điểm đi và điểm đến
Luồng chính	1. Ứng dụng gọi Routes API với phương tiện đã chọn (ô tô/xe máy/đi bộ), kèm intermediates nếu có điểm dừng. 2. Giải mã polyline, lấy mẫu dọc tuyến và tra dữ liệu live TomTom cho các đoạn ứng viên. 3. Chấm điểm tuyến bằng thời gian điều chỉnh theo congestionIndex. 4. Gửi các tuyến ứng viên tới /api/ml/recommend-route; nếu server đề xuất tuyến khác, tự chuyển và thông báo. 5. Hiển thị thời gian, quãng đường, câu trạng thái, sức khỏe tuyến A–F và ước tính nhiên liệu.
Luồng thay thế	4a. Server offline: giữ nguyên tuyến chọn bằng điểm số on-device, không báo lỗi. 1a. Routes API lỗi/quá hạn mức: thông báo nguyên nhân, giữ trạng thái chọn điểm.
Kết quả	Tuyến tối ưu được vẽ trên bản đồ kèm thông tin chi tiết; sẵn sàng bắt đầu dẫn đường.
Yêu cầu liên quan	FR05, FR06, FR07, FR09, FR10, NFR01, NFR03

- (iii) nhóm phát triển đã có sẵn codebase Java ổn định với đầy đủ tính năng, chuyển framework sẽ tạo chi phí rewrite không cần thiết.

Về mặt framework UI, ứng dụng dùng ViewBinding, Material Components (nút pill, chip, bottom sheet) và ViewPager2 cho carousel; đa ngôn ngữ dùng per-app locale của App-Compat.

Bảng 5.4: Đặc tả UC-04: Xem dự báo ML theo đoạn đường.

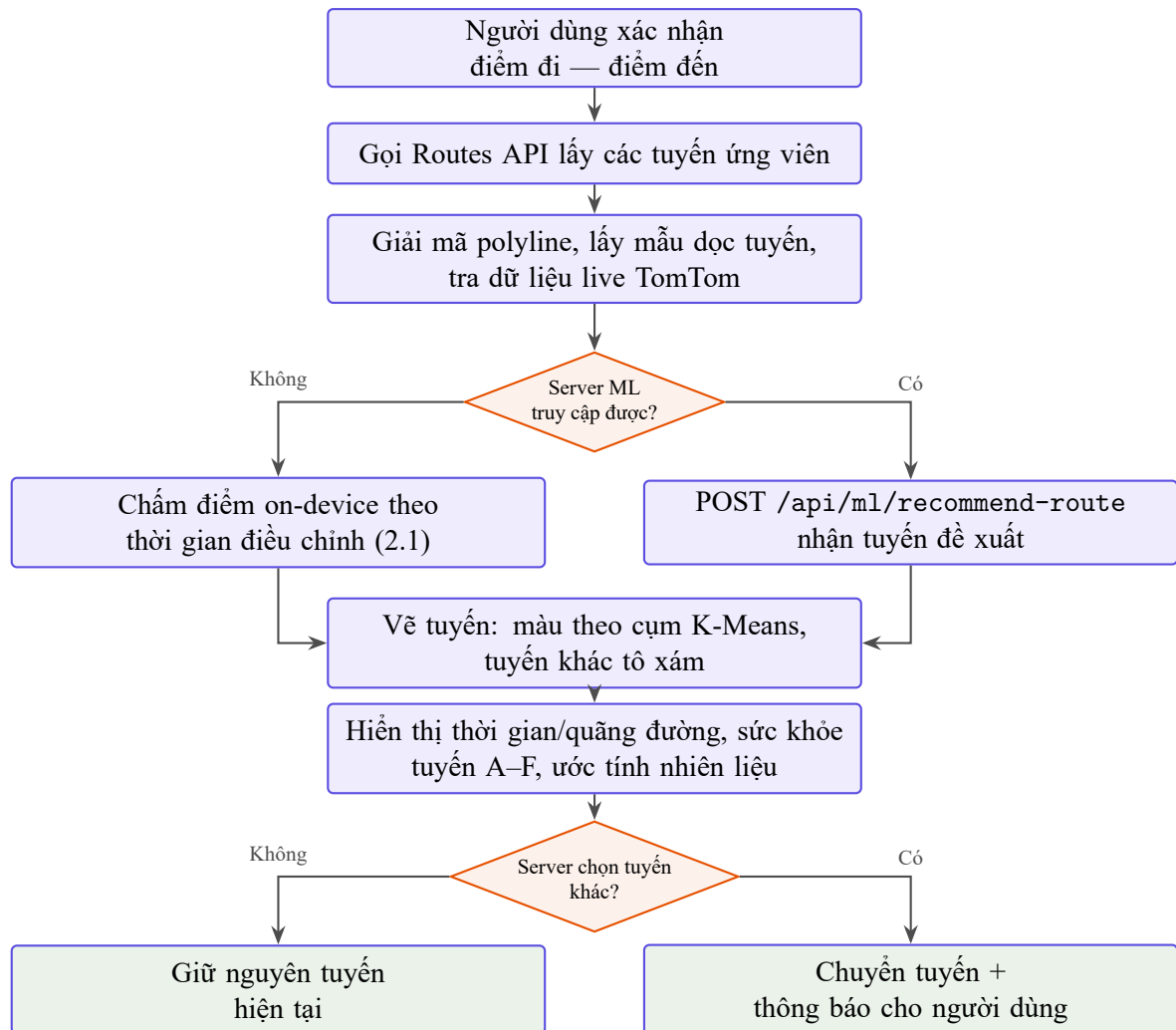
Ca sử dụng	Xem dự báo ML theo đoạn đường (UC-04)
Tác nhân	Người dùng; hệ thống ngoài: server ML, TomTom
Tiền điều kiện	Bottom sheet chi tiết đoạn đang mở
Luồng chính	1. Ứng dụng tổng hợp đặc trưng của đoạn (tốc độ live, dòng tự do, chiều dài, giới hạn tốc độ, loại đường, giờ, ngày). 2. Gọi POST /api/ml/predict với timeout ngắn; lần lượt thử các địa chỉ server mặc định. 3. Server chuẩn hóa, suy luận RF trả tốc độ dự đoán, gán cụm K-Means, đánh giá rủi ro sự cố. 4. Hiển thị thẻ “Dự báo ML”: tốc độ $\approx$, mức kẹt, cụm giao thông, rủi ro.
Luồng thay thế	2a. Không kết nối được server nào: thẻ hiển thị “Server ML không khả dụng”, ứng dụng hoạt động bình thường.
Kết quả	Người dùng thấy dự báo ML cho đoạn đang quan sát.
Yêu cầu liên quan	FR04, FR08, FR10, NFR01, NFR06

Bảng 5.5: Đặc tả UC-09: Huấn luyện lại mô hình.

Ca sử dụng	Huấn luyện lại mô hình (UC-09)
Tác nhân	Quản trị
Tiền điều kiện	Đã cài môi trường Python và các thư viện theo requirements.txt
Luồng chính	1. Cập nhật snapshot mới vào thư mục dữ liệu. 2. Chạy python train_model.py: đọc, làm sạch, join thời tiết, huấn luyện RF, copy mô hình gom cụm, xuất speed_rf.joblib, cluster_model.json, meta.json kèm chỉ số đánh giá. 3. Khởi chạy lại uvicorn app:app; kiểm tra /health xác nhận mô hình đã nạp kèm số liệu MAE/R ² .
Luồng thay thế	2a. Dữ liệu quá ít (< 100 mẫu): dừng và báo lỗi thay vì huấn luyện mô hình kém tin cậy.
Kết quả	Mô hình mới sẵn sàng phục vụ; ứng dụng có thể cập nhật cluster_model.json tương ứng.
Yêu cầu liên quan	FR08, NFR06

5.2.2. Back-end (server ML)

Lựa chọn: FastAPI + scikit-learn thay vì hai phương án cân nhắc là Flask và Node.js/Express [17]. So với Flask, FastAPI có sẵn xác thực schema bằng Pydantic, tự sinh Swagger UI (giảm công sức tài liệu API) và hỗ trợ async. So với kiến trúc Node.js/Express gọi service Python (như đề án gốc), việc chạy FastAPI cùng một tiến trình với mô hình scikit-learn loại bỏ một lớp mạng trung gian, giảm độ trễ và số điểm lỗi, đồng thời vẫn giữ đúng *giao diện API* đã thiết kế trong báo cáo đề án. Trình tự suy luận trong server: chuẩn hóa đặc trưng → Random Forest dự đoán tốc độ → K-Means JSON gán cụm → luật phát hiện sự cố → tổng hợp kết quả trả về.



Hình 5.3: Sơ đồ hoạt động của luồng tìm đường kết hợp dự báo ML.

5.3. Thiết kế

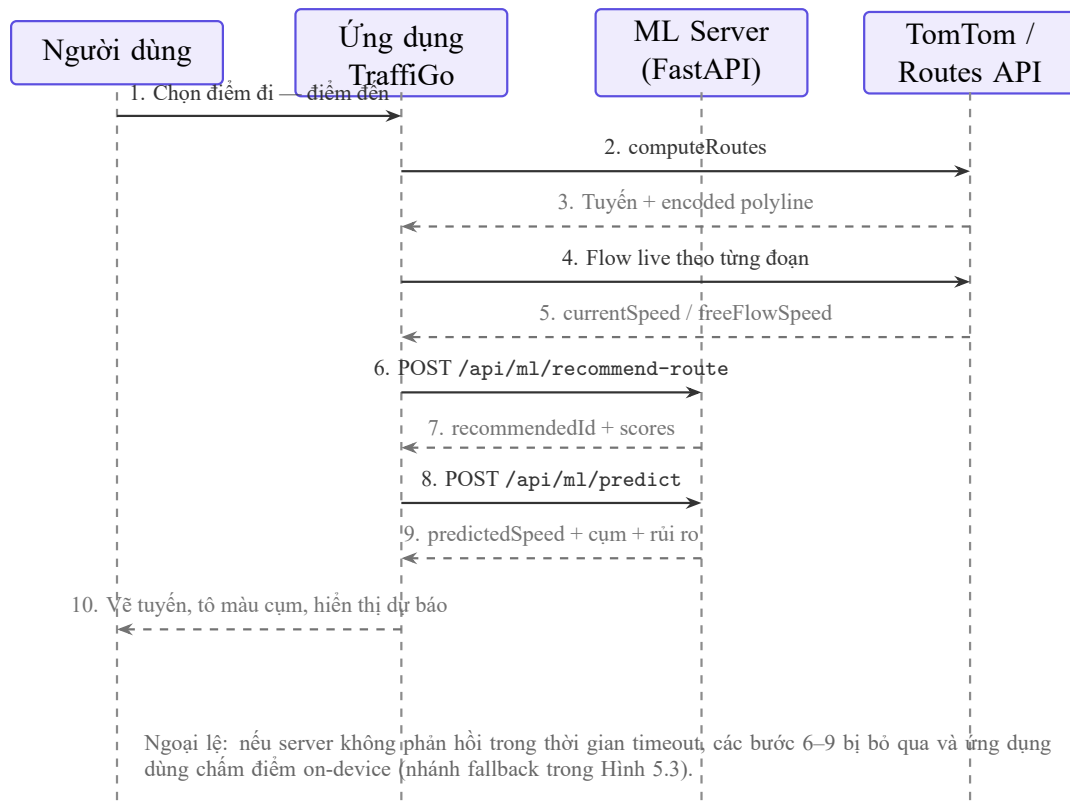
5.3.1. Kiến trúc hệ thống

Hình 5.5 trình bày kiến trúc tổng thể ba tầng của hệ thống.

Kiến trúc được tổ chức sao cho *mô hình là tài sản dùng chung*: tệp `cluster_model.json` xuất từ pipeline được đặt vào cả `assets/` của ứng dụng và `models/` của server, đảm bảo một đoạn luôn được gán về cùng một cụm bất kể nơi suy luận. Tầng ứng dụng giao tiếp với tầng dịch vụ chỉ qua HTTP JSON có timeout ngắn, cho phép hai bên triển khai và cập nhật độc lập.

5.3.2. Thiết kế cơ sở dữ liệu

Dữ liệu người dùng và dữ liệu cộng đồng lưu trên Firebase Realtime Database theo cấu trúc cây JSON Hình 5.6; dữ liệu huấn luyện lưu dạng CSV phẳng theo schema 40 cột (Bảng 5.7).



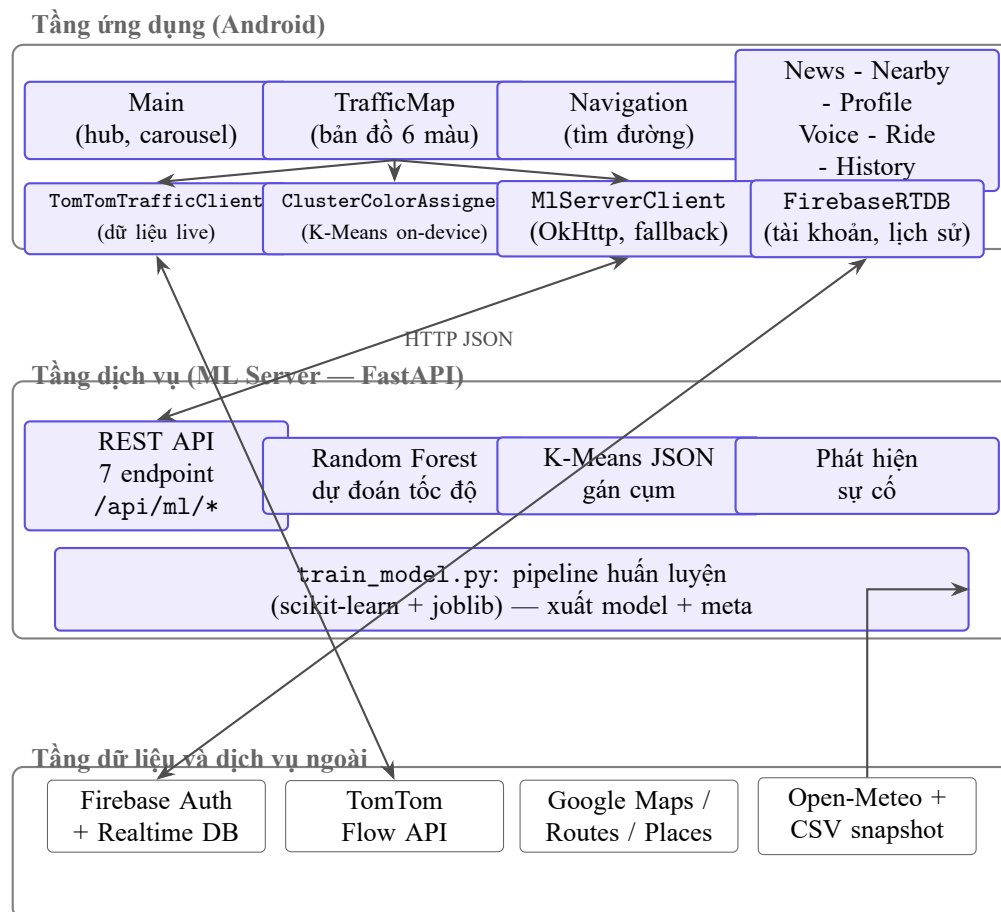
Hình 5.4: Sơ đồ tuần tự luồng tìm đường kết hợp dự báo ML.

Bảng 5.6 mô tả chi tiết mục đích từng đường dẫn cùng quy tắc truy cập đề xuất; việc tách quyền theo uid đảm bảo người dùng chỉ đọc/ghi dữ liệu của chính mình, còn nhánh incidents cho phép mọi người dùng đã xác thực cùng báo và đọc sự cố cộng đồng.

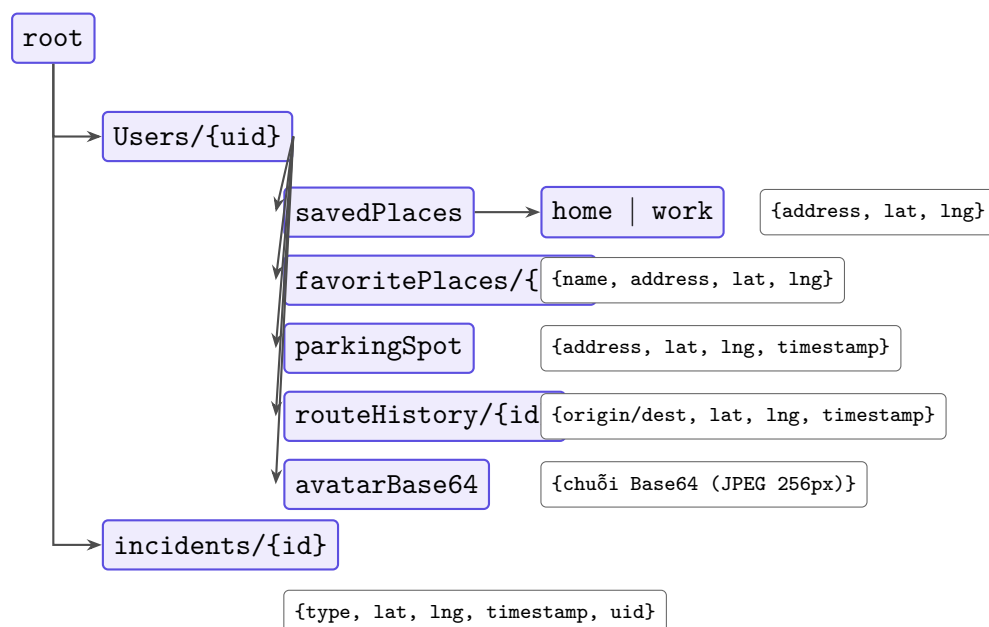
5.3.3. Thiết kế chi tiết bên trong

Suy luận gom cụm on-device (ClusterColorAssigner)

Lớp này đóng gói toàn bộ mô hình gom cụm trên thiết bị. Khi khởi tạo, tệp `cluster_model.json` được đọc một lần trên luồng nền. Với mỗi đoạn, 9 đặc trưng được tổng hợp đúng công thức pipeline, chuẩn hóa theo (2.4) rồi tìm tâm cụm gần nhất theo khoảng cách Euclid (2.1); thứ hạng nghiêm trọng của cụm được ánh xạ thành 6 màu từ đỏ đậm (kẹt nặng nhất) tới xanh đậm (thoáng nhất). Lớp thành thật dữ liệu (F3): đoạn có $\text{freeFlowSpeed} \leq 0$ được tô xám “thiếu dữ liệu” thay vì suy đoán thành xanh — tránh tạo ấn tượng sai rằng đường thoáng.



Hình 5.5: Kiến trúc tổng thể ba tầng của hệ thống.



Hình 5.6: Cấu trúc dữ liệu người dùng trên Firebase Realtime Database.

Bảng 5.6: Mô tả đường dẫn dữ liệu và quy tắc truy cập Firebase.

Đường dẫn	Mục đích	Quy tắc truy cập đề xuất
Users/{uid}/savedPlaces/{home work}	Nhà, công ty phục vụ carousel và phím tắt chỉ đường	Đọc/ghi khi auth.uid == {uid}
Users/{uid}/favoritePlaces	Danh sách địa điểm yêu thích tự đặt tên	Đọc/ghi khi auth.uid == {uid}
Users/{uid}/parkingSpot	Vị trí đỗ xe (đơn, ghi đè mỗi lần lưu)	Đọc/ghi khi auth.uid == {uid}
Users/{uid}/routeHistory	Lịch sử lộ trình, phục vụ “trải lại tuyến này”	Đọc/ghi khi auth.uid == {uid}
Users/{uid}/avatarBase64	Ảnh đại diện mã hóa Base64 (JPEG 256px)	Đọc/ghi khi auth.uid == {uid}
incidents/{id}	Sự cố cộng đồng (kẹt, tai nạn, ngập...), tự hết hạn 2 giờ	Đọc/ghi khi auth != null

Bảng 5.7: Tóm tắt schema tập dữ liệu huấn luyện (40 cột, ví dụ các cột chính).

Vị trí	Tên cột	Ý nghĩa	Kiểu
0–1	segmentId, name_vn	Định danh đoạn (hash), tên đường	text
2–5	lat/lon_start, lat/lon_end	Tọa độ hai đầu đoạn	float
6–19	laneCount, speedLimit, ...	Hạ tầng, hình học, độ dốc, cao độ	float/text
20–23	timeStamp, dayOfWeek, hourOfDay, dayType	Ngữ cảnh thời gian (yyMMddHHmm)	int/text
24–28	currentSpeed, freeFlowSpeed, jamFactor, incidentFlag, congestionIndex	Giao thông động (lỗi của bài toán)	float
29–34	trafficVolume, LOS, occupancy, crossTime, speedLimitRatio, relativeCongestionIndex	Chỉ số suy diễn phục vụ phân tích	float/text
35–39	route_distance/duration/weight, congestion_levels, map_match_confidence	Điều hướng và độ tin cậy	float/JSON

Định tuyến né kẹt (PathFinder)

Đồ thị được xây dựng với các nút là tọa độ đầu–cuối của đoạn; thuật toán Dijkstra tối ưu theo *thời gian* thay vì khoảng cách, trọng số mỗi cạnh nhân thêm hệ số phạt kẹt:

$$w(s) = \frac{d(s)}{v(s)} \cdot p(\text{congestionIndex}(s)), \quad p(c) = 1 + 0,6 \cdot \frac{0,85 - \text{clip}(c, 0,30, 0,85)}{0,85 - 0,30}, \quad (5.1)$$

trong đó $d(s)$ là độ dài, $v(s)$ là tốc độ hiện tại của đoạn s . Hệ số phạt tối đa 1,6 giúp thuật toán ưu tiên đoạn thoát mà không dẫn vòng vô lý; giá trị `congestionIndex` ≤ 0 (không rõ dữ liệu) không bị phạt để tránh bóp méo tuyến.

Client ML (MLServerClient)

Client OkHttp bắt đồng bộ với timeout ngắn (kết nối 1,5s, đọc 4s) nhằm không bao giờ khựng UI; danh sách địa chỉ ứng viên được thử lần lượt (10.0.2.2 cho trình giả lập chuẩn, 127.0.0.1 khi dùng adb `reverse`, hoặc URL người dùng cấu hình);¹ mọi callback được đưa về UI thread và tự bỏ qua khi activity đã hủy. Cơ chế *generation counter* trong `NavigationActivity` loại bỏ các callback trả về muộn khi người dùng đã reset giữa chừng — tránh trạng thái “tuyến cũ sống lại”.

Tối ưu bản đồ (TrafficMapActivity)

Nạp hình học 23.417 đoạn chạy trên luồng nền và công bố kết quả qua các biến `volatile` (tránh đọc thấy trạng thái dở dang); chỉ mục không gian dạng lưới ô $\approx 900\text{m}$ giúp tra đoạn theo khung nhìn trong $O(1)$ thay vì quét tuyến tính; dữ liệu live có TTL 3 phút; gọi TomTom chỉ khi `zoom` ≥ 14 và tối đa 150 đoạn/lần render, ưu tiên gần tâm màn hình — cân bằng giữa độ tươi dữ liệu và quota API.

Tin tức giao thông (TrafficNewsActivity)

Tin tức tổng hợp từ năm báo điện tử (VnExpress, Thanh Niên, Tuổi Trẻ, Dân Trí, VietnamNet) $\times$ năm danh mục, tất cả 25 URL RSS đều được xác minh thật. Ba điểm thiết kế đáng chú ý:

- (i) kết quả từ các nguồn về không đồng bộ nên danh sách được *sắp xếp lại theo thời gian phát* mỗi khi một nguồn phản hồi, tránh bài báo bị “bó cụm” theo thứ tự nguồn;
- (ii) phân tích ngày phát phải thử ba định dạng khác nhau vì các báo không thống nhất (RFC822 năm 4 chữ số, RFC822 năm 2 chữ số, kiểu Mỹ M/d/yyyy);
- (iii) ảnh đại diện bài viết lấy từ hai vị trí khác nhau tùy nguồn (thẻ `<img>` trong mô tả hoặc thẻ `<enclosure>`).

Chức năng “nghe bài” scrape nội dung qua WebView với bộ chọn CSS riêng cho từng báo và ba lớp fallback, có timeout tự phục hồi 6 giây.

¹10.0.2.2 là địa chỉ đặc biệt trên trình giả lập Android chính thức, ánh xạ về loopback (127.0.0.1) của máy phát triển; với trình giả lập MuMu/LDPlayer hoặc thiết bị thật, dùng adb `reverse` hoặc IP LAN.

Trợ lý giọng nói (VoiceAssistantActivity)

Trợ lý dùng Gemini với cơ chế *function calling bắt buộc* (`tool_config.mode = ANY`): giọng nói chuyển thành văn bản, Gemini trả về một `VoiceIntent{action, args}` có cấu trúc, ứng dụng điều phối tới đúng tính năng (dẫn đường, xem kết xe, đọc tin, gọi xe, chia sẻ vị trí, mở bản đồ ngoại tuyến...).² Nút micro được gắn toàn cục qua `ActivityLifecycleCallbacks` — bất kỳ màn hình nào có view id `btnVoiceMic` đều tự có trợ lý mà không cần lập code. Giọng phản hồi dùng TTS tiếng Việt; thất bại khởi tạo TTS (thường gặp trên trình giả lập) được ghi nhận và hiển thị thông báo thay vì im lặng.

5.3.4. Thiết kế giao diện

Giao diện được thiết kế theo một hệ định danh trực quan thống nhất áp dụng cho mọi màn hình:

- (i) màu chủ đạo tím thương hiệu #5C4FE0 cho nút hành động chính, chip được chọn và các điểm nhấn;
- (ii) nền sáng, thẻ bo góc lớn 16–28dp, nút dạng pill;
- (iii) header trắng với icon tối và tiêu đề in đậm;
- (iv) chip outline khi chưa chọn, fill tím khi chọn;
- (v) bảng màu trạng thái giao thông cố định sáu mức (đỏ đậm → xanh đậm).

Bộ thiết kế này đồng bộ toàn bộ 20 màn hình của ứng dụng, giúp trải nghiệm liền mạch như một sản phẩm duy nhất. Một số màn hình tiêu biểu được minh họa trong mục kết quả ứng dụng (mục 6.1.8, Chương 6).

5.4. Kết chương

Chương này đã phân tích ba luồng nghiệp vụ, mô hình hóa tương tác bằng sơ đồ use case kèm sáu đặc tả chi tiết cùng sơ đồ hoạt động và sơ đồ tuần tự cho luồng phức tạp nhất, lựa chọn công nghệ front-end/back-end kèm so sánh phương án, và thiết kế hệ thống ở bốn mức: kiến trúc ba tầng, cơ sở dữ liệu, chi tiết bên trong và giao diện. Các thiết kế này là bản vẽ thi công cho chương hiện thực kế tiếp.

²Khóa Gemini trên gói miễn phí giới hạn khoảng 20 yêu cầu/ngày cho mô hình flash; khi vượt hạn mức, lỗi `RESOURCE_EXHAUSTED` được bắt và hiển thị thông báo rõ cho người dùng thay vì im lặng.

CHƯƠNG 6

HIỆN THỰC VÀ KIỂM THỬ

6.1. Hiện thực

6.1.1. Pipeline dữ liệu

Bộ công cụ thu thập được viết bằng Python, gọi TomTom Flow API theo tọa độ hai đầu từng đoạn trong khung giờ định sẵn và lưu snapshot theo tên tệp `traffic_hcm_<ngày>_<giờ>.csv`. Trong giai đoạn 09/03–15/04/2026, hệ thống thu được **171 snapshot** (khoảng 170.000 dòng, 917 đoạn duy nhất). Pipeline làm sạch: loại dòng thiếu cột lỗi, chuẩn hóa kiểu dữ liệu, giới hạn miền giá trị hợp lý (tốc độ trong (0; 150) km/h, giờ trong [0; 23], ngày trong [0; 6]) và khử trùng lặp theo `segmentId`; kết quả còn **149.023 mẫu**. Dữ liệu thời tiết ngày cho TP.HCM (38 ngày) được lấy từ Open-Meteo Archive và join theo ngày được suy ra từ `timeStamp`, phủ **100% mẫu**. Bảng 6.1 thống kê phân bố của tập dữ liệu theo ngữ cảnh thời gian; có thể thấy giờ cao điểm chiếm gần một phần ba số mẫu và có `congestionIndex` trung bình thấp hơn (kẹt hơn theo quy ước của đề tài) so với ngoài giờ cao điểm — tín hiệu hợp lý cho thấy dữ liệu phản ánh đúng nhịp sinh hoạt giao thông.

Bảng 6.1: Phân bố tập dữ liệu theo ngữ cảnh thời gian ($n = 169.897$ mẫu).

Nhóm	Số mẫu	Tỉ lệ	<code>congestionIndex</code> TB
Giờ cao điểm (7–9h, 17–19h)	54.670	32,2%	0,751
Ngoài giờ cao điểm	115.227	67,8%	0,809
Ngày thường	125.167	73,7%	0,781
Cuối tuần	44.730	26,3%	0,818

Thời gian chạy toàn bộ nhánh Feature Selection (từ đọc dữ liệu tới sinh hình ảnh phân tích) khoảng 70 giây trên máy cá nhân.

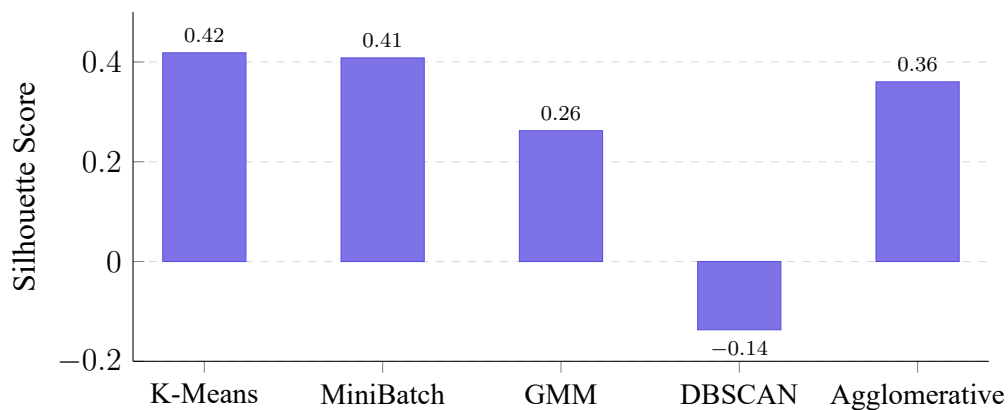
6.1.2. Kết quả lựa chọn thuộc tính và gom cụm

Nhân trạng thái giao thông tham chiếu phục vụ đánh giá ngoại bộ được xây dựng từ tốc độ tương đối và ngưỡng LOS. Hai phương pháp lựa chọn thuộc tính (Mutual Information và Random Forest Importance) được chạy song song; đánh giá top- k cho thấy tập 10 thuộc tính là cân bằng tốt nhất giữa thông tin giữ được và số chiều. Nhánh Feature Selection (10 thuộc tính) sau đó được so sánh với nhánh tìm tham số giữ nguyên 40 thuộc tính, trên năm thuật toán với k khảo sát trong [2; 12] (Elbow, BIC/AIC cho GMM, k-distance cho DBSCAN).

Bảng 6.2 tổng hợp kết quả trên nhánh Feature Selection với $k = 6$; Hình 6.1 trực quan hóa Silhouette Score của năm thuật toán.

Bảng 6.2: Kết quả đánh giá gom cụm trên nhánh Feature Selection ($k = 6$).

Mô hình	Silhouette↑	DBI↓	CH↑	ARI↑	NMI↑	FMI↑
K-Means	0,4183	0,8063	147574,6	0,3926	0,4793	0,4573
MiniBatchKMeans	0,4081	0,9040	139501,5	0,5013	0,5196	0,4970
GMM	0,2622	1,5711	62459,1	0,4271	0,4665	0,4018
DBSCAN	-0,1368	0,7449	2838,4	0,3338	0,4274	0,6382
Agglomerative	0,3601	0,9273	133440,5	0,3692	0,4932	0,5093



Hình 6.1: So sánh Silhouette Score của năm thuật toán trên nhánh Feature Selection.

K-Means đạt Silhouette cao nhất (0,4183) và Calinski–Harabasz vượt trội (147574,6) chứng tỏ các cụm phân tách tốt; DBSCAN tuy DBI thấp nhất nhưng tạo rất ít cụm với lượng nhiễu lớn (CH chỉ 2838,4) nên không phù hợp; GMM bị chi phối bởi số chiều sau one-hot. So sánh hai K-Means tốt nhất của hai nhánh: Feature Selection (10 thuộc tính) đạt Silhouette 0,4183 so với 0,1310 của nhánh 40 thuộc tính, và DBI 0,8063 so với 1,8745 — xác nhận hiệu quả của lựa chọn thuộc tính. **K-Means $k = 6$ nhánh Feature Selection** được chọn là mô hình cuối cùng. Bảng 6.3 đối chiếu trực tiếp hai K-Means tốt nhất của hai nhánh thực nghiệm: với chỉ 10 thuộc tính (so với 40), nhánh Feature Selection cải thiện Silhouette gấp 3,2 lần, giảm Davies–Bouldin từ 1,8745 xuống 0,8063 và tăng Calinski–Harabasz gần 7 lần — khẳng định rằng chất lượng gom cụm phụ thuộc vào *sự liên quan* của thuộc tính nhiều hơn *số lượng* thuộc tính, đồng thời mô hình gọn hơn cũng dễ nhúng lên thiết bị di động hơn.

Sáu cụm được diễn giải theo congestionIndex trung bình và thứ hạng nghiêm trọng (Bảng 6.4, số liệu lấy trực tiếp từ tệp mô hình đã xuất). Thứ hạng 0 là cụm kẹt nặng nhất và được tô màu đỏ đậm trên bản đồ.

Bảng 6.3: So sánh hai mô hình K-Means tốt nhất từ hai nhánh thực nghiệm ($k = 6$).

Chỉ tiêu	Feature Selection	Tìm tham số
Số thuộc tính đầu vào	10	40
Silhouette Score $\uparrow$	0,4183	0,1310
Davies–Bouldin Index $\downarrow$	0,8063	1,8745
Calinski–Harabasz Index $\uparrow$	147574,6	21906,92

Bảng 6.4: Diễn giải sáu cụm giao thông (số liệu từ `cluster_model.json`).

Cụm	congestionIndex TB	Thứ hạng	Trạng thái diễn giải
2	0,6195	0	Kẹt nặng: tốc độ rơi sâu so với dòng tự do
5	0,7661	1	Kẹt cục bộ, dòng xe chậm
1	0,7876	2	Đông đúc, lưu lượng cao
4	0,8250	3	Trung bình, bắt đầu chậm lại
0	0,9689	4	Thoảng, lưu thông gần dòng tự do
3	1,0000	5	Thoảng hoàn toàn / bất thường dữ liệu

6.1.3. Xuất mô hình và nhúng lên ứng dụng

Mô hình được xuất qua công cụ `export_cluster_model.py` thành `cluster_model.json` gồm: danh sách 9 thuộc tính, tham số `scaler_mean/scaler_scale`, sáu vector tâm cụm, ánh xạ `cluster_severity_rank` và `cluster_mean_congestion_index`. Tập này được đặt song song ở hai nơi: `assets/` của ứng dụng (suy luận on-device qua `ClusterColorAssigner`) và `models/` của server (suy luận bằng NumPy) — đảm bảo một đoạn luôn cùng cụm bất kể nơi suy luận.

6.1.4. Server Machine Learning

Mô hình dự đoán tốc độ là **Random Forest Regressor** huấn luyện trên 149.023 mẫu với 9 thuộc tính số, 3 thuộc tính phân loại one-hot và 3 thuộc tính thời tiết; cấu hình **60 cây**, **`min_samples_leaf=6`**, **`max_depth=22`**, **`max_features=sqrt`** — phương án “lite” chủ động rút gọn để vừa bộ nhớ 512 MB của Render free (bản đầy đủ 120 cây đạt MAE 1,84 km/h nhưng chiếm 330 MB RAM khi nạp, vượt hạn mức). Trên tập giữ-out 7.452 mẫu, kết quả đạt Bảng 6.5: MAE 2,19 km/h — tốt hơn gần 4 lần baseline “luôn giả định đường thoáng” (8,25 km/h), $R^2 = 0,919$.

Bảng 6.5: Kết quả mô hình Random Forest dự đoán tốc độ lưu thông (cấu hình lite đã triển khai).

Chỉ số	Random Forest (lite)	Baseline (dự đoán = freeFlow)
MAE (km/h)	2,19	8,25
RMSE (km/h)	2,94	—
R^2	0,919	—
Số mẫu kiểm tra	7.452	7.452

Server FastAPI hiện thực bảy API đúng Bảng 6.6. Ví dụ lời gọi dự đoán và phản hồi thực tế (giờ cao điểm, thứ Sáu):

POST /api/ml/predict

```
{ "segment": { "currentSpeed": 25, "freeFlowSpeed": 60,
  "hourOfDay": 18, "dayOfWeek": 6, "lengthKm": 1.2 } }
```

200 OK

```
{ "prediction": { "predictedSpeed": 45.7, "trafficLevel": "Moderate",
  "congestionIndex": 0.762, "clusterId": 2, "severityRank": 0,
  "incident": { "risk": "MEDIUM", "incident": true,
  "confidence": 0.85,
  "reason": "Tốc độ thực đo thấp hơn dự đoán 21 km/h
  - nghi sự cố/tai nạn" } } }
```

Phát hiện sự cố kết hợp hai tín hiệu:

- (i) luật trên tỉ lệ tốc độ thực đo/dòng tự do (dưới 0,30 $\Rightarrow$ HIGH);
- (ii) sai số giữa tốc độ thực đo và dự đoán — sụt giảm ≥ 15 km/h kèm tỉ lệ dưới 0,5 được đánh dấu nghi sự cố với độ tin cậy tỉ lệ theo mức sụt.

Cơ chế này hoạt động như một detector bất thường ở mức đoạn.

Bảng 6.6: Các API Machine Learning của hệ thống.

Endpoint	Method	Chức năng
/api/ml/predict	POST	Dự đoán giao thông cho một đoạn
/api/ml/predict-batch	POST	Dự đoán nhiều đoạn
/api/ml/recommend-route	POST	Đề xuất tuyến đường tối ưu
/api/ml/compare-routes	POST	So sánh hai tuyến đường
/api/ml/detect-incident	POST	Phát hiện sự cố giao thông
/api/ml/detect-incidents	POST	Phát hiện nhiều sự cố
/api/ml/traffic-status	POST	Tổng hợp trạng thái giao thông
/health	GET	Sức khỏe server, trạng thái nạp model, chỉ số MAE/R ²

6.1.5. Tích hợp vào ứng dụng di động

Ứng dụng tiêu thụ server ML ở hai điểm:

- (i) **thẻ Dự báo ML** trong bottom sheet chi tiết đoạn của bản đồ giao thông — khi người dùng bấm vào đoạn, đặc trưng được gửi tới /api/ml/predict và thẻ tím hiện tốc độ dự đoán, mức kẹt, số cụm và rủi ro sự cố (Hình 6.6);

- (ii) **đề xuất tuyến** sau khi tìm đường — ứng dụng gửi các tuyến ứng viên tới `/api/ml/recommend-route`, server chấm điểm lại theo tốc độ dự đoán, nếu chọn tuyến khác thì ứng dụng tự chuyển kèm thông báo.

Mọi lời gọi đều bắt đồng bộ với timeout ngắn và fallback im lặng về logic on-device.

6.1.6. Cập nhật dữ liệu live định kỳ phía server

Ngoài vai trò phục vụ dự đoán, server ML còn chạy một **bộ cập nhật dữ liệu live** (refresher) trên luồng nền: theo chu kỳ cấu hình (mặc định **15 phút**), server tự gọi TomTom Flow API cho một tập đoạn đường giám sát (mặc định 50 đoạn đầu trong `geometry.csv`) và lưu snapshot mới nhất vào bộ nhớ; kết quả được phục vụ qua endpoint `GET /api/ml/live-status` gồm: trạng thái bộ cập nhật, số chu kỳ đã chạy, số đoạn lấy thành công/thất bại ở chu kỳ gần nhất, thời điểm cập nhật kế tiếp và danh sách các đoạn có dữ liệu mới nhất kèm mốc thời gian.

Thiết kế tham số được cân bằng theo hạn mức miễn phí: với 8 khóa TomTom xoay vòng, 50 đoạn và chu kỳ 15 phút, tổng tiêu thụ ≈ 4.800 lời gọi/ngày, chia đều ≈ 600 lời gọi/khóa/ngày — dư dả so với hạn mức 2.500 lời gọi/khóa/ngày của gói free. Ba biến môi trường cho phép điều chỉnh không cần sửa code (bảng 6.7); khi không đặt, server chạy ngay với cấu hình mặc định an toàn đã cân bằng theo hạn mức free.

Bảng 6.7: Biến cấu hình bộ cập nhật dữ liệu live của server.

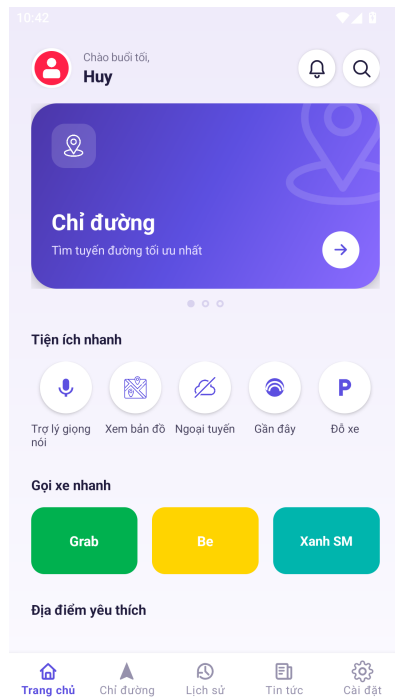
Biến môi trường	Mặc định	Ý nghĩa
TRAFFIGO_REFRESH_DISABLE	(trống = bật)	Đặt 1 để tắt bộ cập nhật (vd. khi không có khóa TomTom)
TRAFFIGO_REFRESH_INTERVAL	900 (giây)	Chu kỳ giữa hai lần quét dữ liệu live
TRAFFIGO_REFRESH_MAX	50	Số đoạn đường giám sát (50 đoạn đầu của <code>geometry.csv</code>)
TOMTOM_KEYS	Đọc từ <code>local.properties</code>	Danh sách khóa API xoay vòng, phân tách bằng dấu phẩy

Trên giao diện ứng dụng, một chip trạng thái hiển thị ngay trên bản đồ “Server ML: đang hoạt động/ngoại tuyến” (Hình 6.2b) — cập nhật khi mở màn hình và sau mỗi lời gọi ML thực tế, để người dùng luôn biết hệ thống ML có sẵn sàng hay không thay vì im lặng.

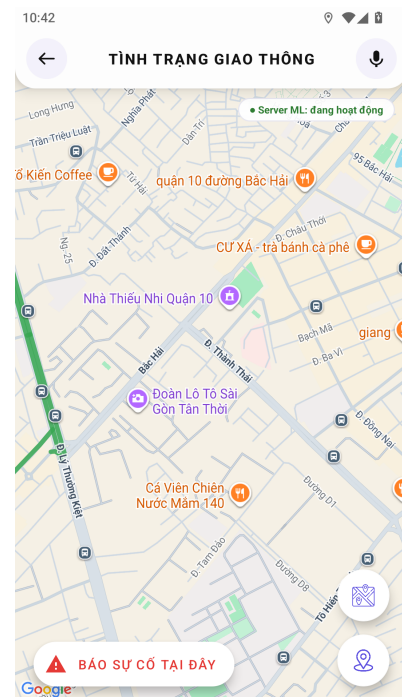
6.1.7. Triển khai server chính thức trên Render Cloud

Server ML chính thức đang chạy trên Render Cloud tại địa chỉ `https://traffigo-server.onrender.com`. Ứng dụng Android đã được cấu hình kết nối tới host này; khi endpoint

/health phản hồi thành công, chip trạng thái trên bản đồ chuyển sang màu xanh và hiển thị “Server ML: đang hoạt động” (Hình 6.2b). Các lời gọi dự báo và đề xuất tuyến sử dụng cùng host này qua HTTPS; nếu dịch vụ không khả dụng, ứng dụng vẫn dùng cơ chế fallback on-device đã trình bày ở trên. Địa chỉ này là môi trường triển khai chính thức của đồ án, khác với địa chỉ localhost chỉ dùng khi phát triển và kiểm thử cục bộ.



(a) Trang chủ sau khi đồng bộ giao diện



(b) Chip trạng thái server ML trên bản đồ

Hình 6.2: Phản hồi trạng thái hệ thống cho người dùng.

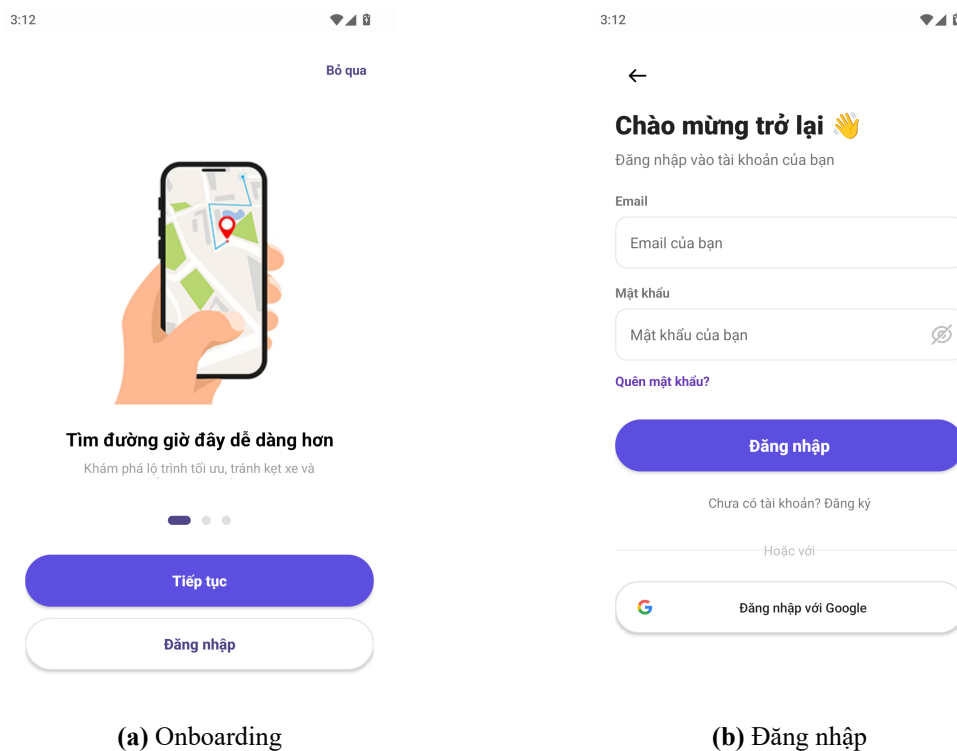
6.1.8. Kết quả ứng dụng

Phần này trình bày kết quả hiện thực của ứng dụng TraffiGo qua các màn hình chính, tất cả được chụp trực tiếp trên trình giả lập với giao diện tiếng Việt.

Khởi động và đăng nhập. Ứng dụng mở bằng màn hình onboarding ba trang giới thiệu (Hình 6.3a) với nút “Bỏ qua” đưa thẳng tới đăng nhập; màn đăng nhập (Hình 6.3b) hỗ trợ email/mật khẩu và Google Sign-In qua Firebase Authentication — toàn bộ luồng đã được kiểm chứng end-to-end với tài khoản thực.

Trang chủ. Hình 6.4 trình bày hub trung tâm với carousel chỉ đường (Chỉ đường / Về nhà / Đến công ty), hàng tiện ích nhanh (trợ lý giọng nói, bản đồ, ngoại tuyến, gần đây, đỗ xe), gọi xe nhanh tới Grab/Be/Xanh SM và danh sách địa điểm yêu thích; thanh điều hướng dưới dẫn tới bốn nhóm chức năng chính.

Bản đồ giao thông theo cụm. Hình 6.5 là màn hình lõi của hệ thống: các đoạn đường được tô màu theo cụm K-Means (xanh = thoáng, đỏ = kẹt) dựa trên dữ liệu live TomTom; đoạn chưa có dữ liệu tô xám. Khi bấm vào một đoạn (Hình 6.6), bottom sheet hiện tốc độ



Hình 6.3: Luồng khởi động và đăng nhập của ứng dụng.

live và trạng thái, kèm **thể “Dự báo ML (server)”** — kết quả tích hợp server: tốc độ dự đoán ≈ 28 km/h, mức trung bình, cụm giao thông #0, rủi ro sự cố THẤP.

Chỉ đường né kẹt. Luồng tìm đường bắt đầu từ màn “Bắt đầu hành trình” (Hình 6.7a): điểm đi mặc định là vị trí GPS hiện tại (đã reverse-geocode sang địa chỉ tiếng Việt), có thể chọn “Vị trí hiện tại”, nhà/công ty đã lưu hoặc chạm bản đồ; chọn phương tiện ô tô/xe máy/đi bộ và tùy chọn tránh phí (Hình 6.7b — hỗ trợ thêm điểm dừng trung gian).

Hình 6.8 thể hiện kết quả chấm điểm tuyến: với tuyến 16 phút/5,7 km, hệ thống phát hiện **đoạn tắc nghẽn nặng** trên tuyến và cảnh báo “ước tính thêm ~ 25 phút”, gán **sức khỏe tuyến F**; dòng “TraffiGo chọn tuyến ít tắc hơn tuyến nhanh nhất của Google” xuất hiện khi thuật toán chọn tuyến dài hơn nhưng thông thoáng hơn dựa trên trọng số phạt kẹt (5.1) — minh chứng trực quan cho giá trị của kết quả gom cụm trong định tuyến.

Tin tức và địa điểm lân cận. Hình 6.9a tổng hợp tin giao thông từ năm báo điện tử (VnExpress, Thanh Niên, Tuổi Trẻ, Dân Trí, VietNamNet) với chip lọc danh mục; bài viết có thể được đọc to bằng TTS với tốc độ điều chỉnh. Hình 6.9b tìm địa điểm quanh vị trí GPS theo danh mục (Xăng, ATM, Ăn uống, Bệnh viện...) với khoảng cách thực tế.

Hồ sơ, lịch sử và bản đồ ngoại tuyến. Hình 6.10a quản lý thông tin cá nhân (tên, số điện thoại, avatar lưu Base64 vào Realtime Database); Hình 6.10b lưu lại các chuyến đi để *trải lại tuyến này* chỉ với một chạm; Hình 6.10c là danh sách vùng bản đồ đã lưu (tên, ngày lưu, dung lượng) dùng khi không có kết nối mạng.



Hình 6.4: Màn hình Trang chủ của TraffiGo.

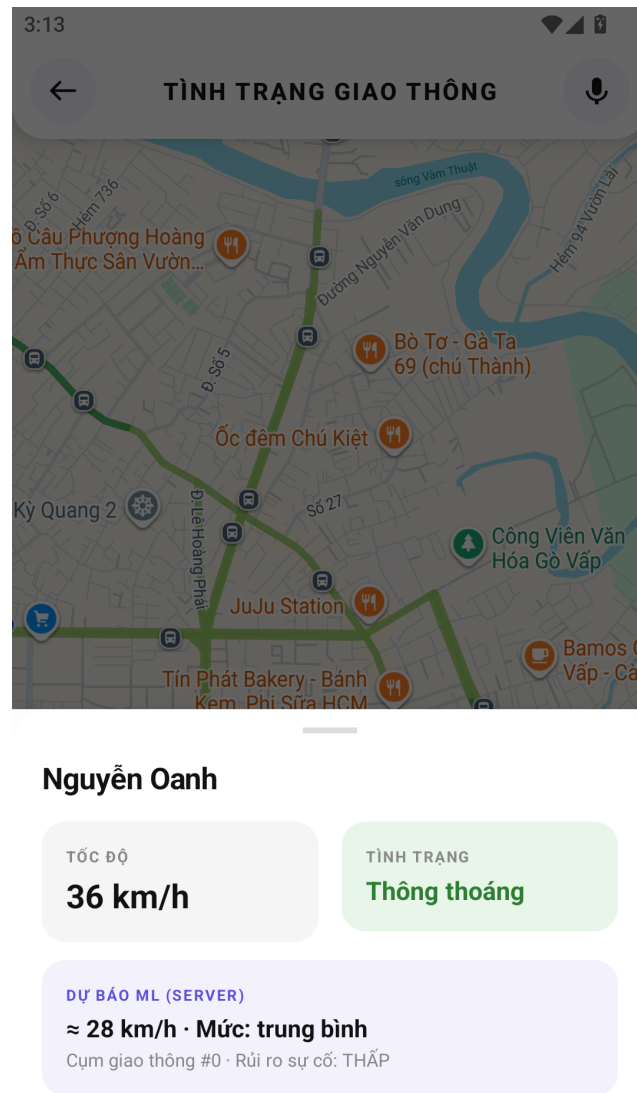
6.2. Ứng dụng kết quả gom cụm vào ứng dụng TraffiGo

Phần này trả lời câu hỏi trung tâm của đề tài: *kết quả gom cụm được đưa vào ứng dụng ở những đâu và mang lại giá trị gì cho người dùng?* Sáu điểm tích hợp chính được tóm tắt trong bảng 6.8 và được phân tích chi tiết ở các mục tiếp theo, kèm ví dụ tính toán bằng số liệu thật từ mô hình.

6.2.1. Tô màu trạng thái giao thông thời gian thực (on-device)

Đây là ứng dụng trọng tâm. Với mỗi đoạn đường hiển thị trên bản đồ, ứng dụng thực hiện đúng bốn bước của mô hình nghiên cứu, toàn bộ trên thiết bị (Hình 6.5):

Bước 1. Lấy dữ liệu live: gọi TomTom Flow cho đoạn, nhận `currentSpeed` và `freeFlowSpeed`;

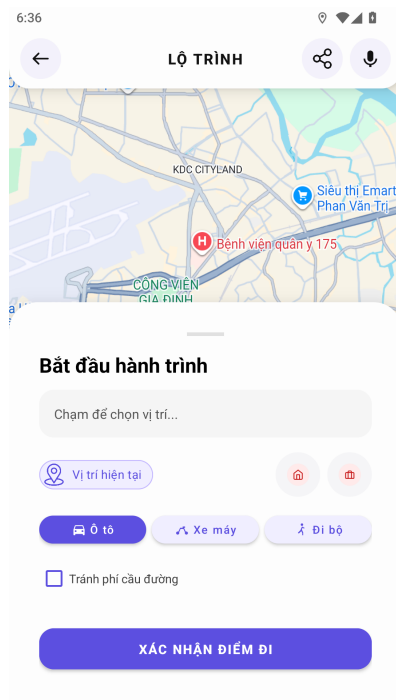


Hình 6.6: Bottom sheet chi tiết đoạn với thẻ Dự báo ML từ server.

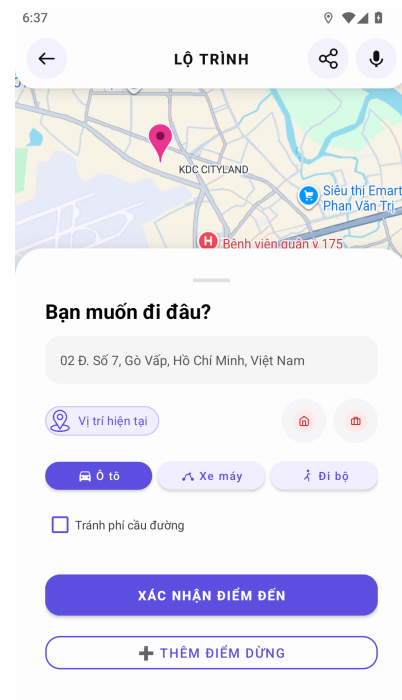
- Bước 2. Tổng hợp 9 đặc trưng** đúng công thức pipeline (tỉ lệ tốc độ, thời gian vượt đoạn, lưu lượng ước lượng, congestionIndex, độ chiếm dụng, chỉ số tương đối, dòng tự do, chiều dài, độ cong);
- Bước 3. Chuẩn hóa** bằng scaler và **tìm tâm cụm gần nhất** theo khoảng cách Euclid;
- Bước 4. Ánh xạ màu** theo thứ hạng nghiêm trọng của cụm: đỏ đậm (kẹt nặng nhất) → cam → vàng → xanh nhạt → xanh đậm (thoáng nhất).

Ví dụ với một đoạn dài 1,2 km, TomTom báo đang chạy 25 km/h trong khi dòng tự do 60 km/h (giờ cao điểm). Bảng 6.9 là 9 đặc trưng tính được; bảng 6.10 là khoảng cách chuẩn hóa tới sáu tâm cụm từ chính tệp `cluster_model.json` nhúng trong ứng dụng.

Kết quả: đoạn rơi vào **cụm 2** — cụm có congestionIndex trung bình thấp nhất (0,62) và thứ hạng 0 — được tô **đỏ đậm**, đúng trực giác “đang chạy bằng một nửa tốc độ dòng tự do là kẹt nặng”. Ngược lại, một đoạn chạy 52/55 km/h thuộc **cụm 0** (thứ hạng 4) tô



(a) Bắt đầu hành trình



(b) Chọn điểm đến, thêm điểm dừng

Hình 6.7: Luồng chọn điểm đi — điểm đến.**Bảng 6.9:** Ví dụ tính đặc trưng cho một đoạn (25/60 km/h, 1,2 km).

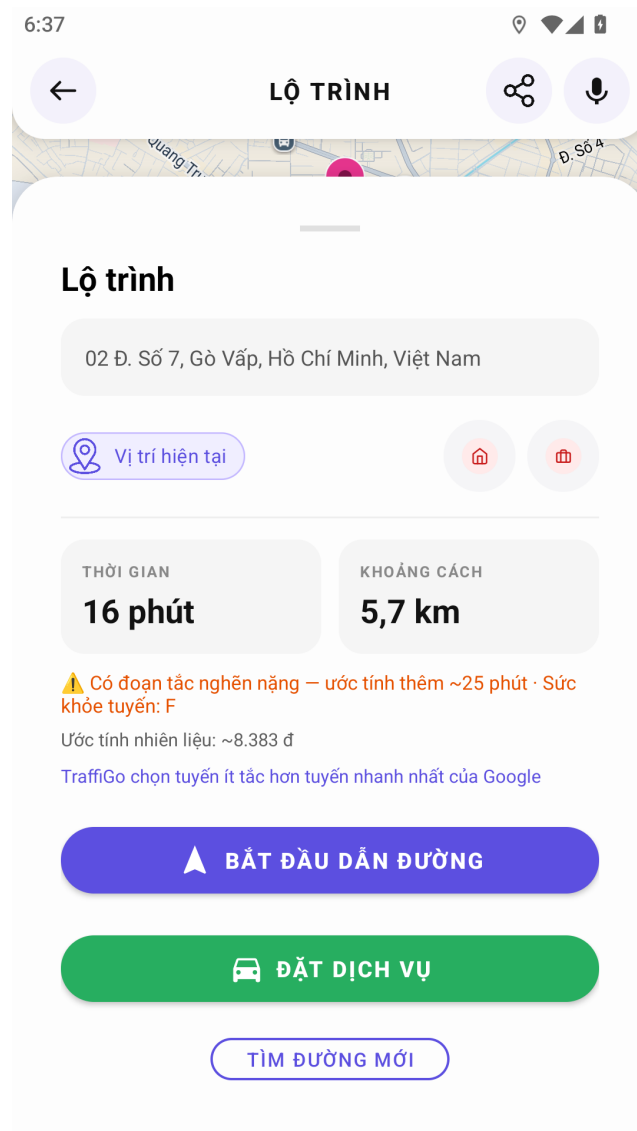
Đặc trưng	Giá trị
speedLimitRatio	0,417
crossTime (giờ)	0,173
trafficVolume (xe/giờ)	1.410
congestionIndex	0,417
occupancy (%)	58,3
relativeCongestionIndex	0,583
freeFlowSpeed (km/h)	60
lengthKm	1,2
curvatureIndex	0,05

Cụm	Khoảng cách	Thứ hạng
0	7,72	4
1	5,65	2
2	3,56 (gần nhất)	0 — đỏ đậm
3	8,32	5
4	10,17	3
5	5,14	1

Bảng 6.10: Kết quả phân loại hai đoạn đối chiếu (suy luận bằng cluster_model.json).

Đoạn	Live/FreeFlow	Cụm	Thứ hạng	Màu hiển thị
Kẹt giờ cao điểm	25/60 km/h	2	0	Đỏ đậm (kẹt nặng nhất)
Thoáng ngoài giờ	52/55 km/h	0	4	Xanh (thoáng)

xanh. Việc suy luận chỉ gồm một phép nhân vector 9×9 và sáu phép tính khoảng cách, thực hiện trên luồng nền với chi phí không đáng kể, cho phép tô màu tức thì cho hàng trăm đoạn trong khung nhìn.



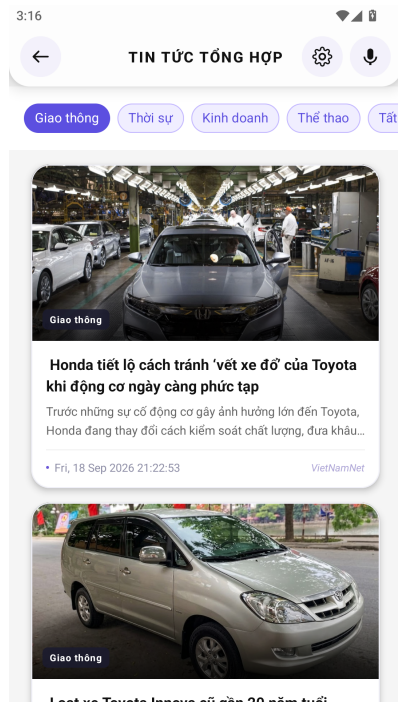
Hình 6.8: Thông tin tuyến: thời gian, quãng đường, cảnh báo tắc, sức khỏe tuyến, ước tính nhiên liệu.

6.2.2. Định tuyến né khu vực kẹt

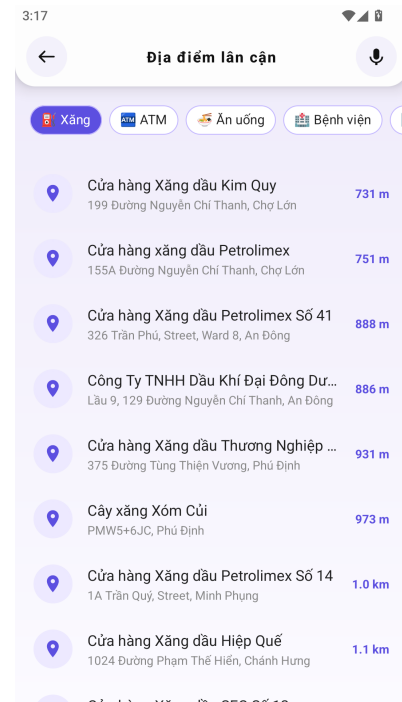
Trọng số mỗi đoạn trong thuật toán Dijkstra là thời gian di chuyển nhân hệ số phạt theo công thức (5.1). Với đoạn trong ví dụ trên ($\text{congestionIndex} = 0,417$), hệ số phạt là

$$p = 1 + 0,6 \cdot \frac{0,85 - 0,417}{0,85 - 0,30} \approx 1,47,$$

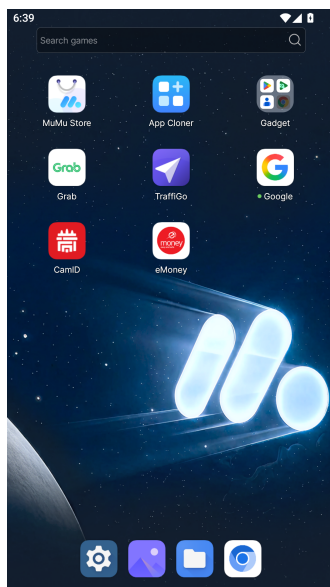
tức thời gian “giá” của đoạn này bị tính gấp 1,47 lần thực tế — thuật toán sẽ ưu tiên đoạn thoáng (hệ số 1,0) trừ khi phải đi vòng quá nhiều. Cơ chế này vận hành trong một luồng thống nhất với dữ liệu live (Hình 5.4): kết quả chạy thật trên ứng dụng cho thấy tuyến 16 phút/5,7 km bị phát hiện “có đoạn tắc nghẽn nặng — ước tính thêm ~25 phút”, sức khỏe tuyến F, và ứng dụng tự ghi nhận “TraffiGo chọn tuyến ít tắc hơn tuyến nhanh nhất của Google” (Hình 6.8) — minh chứng trực quan rằng kết quả gom cụm thực sự thay đổi



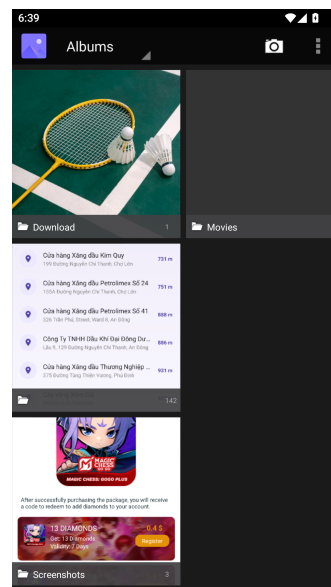
(a) Tin tức tổng hợp



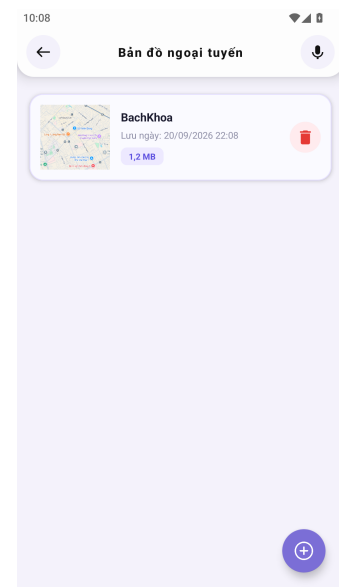
(b) Địa điểm lân cận

Hình 6.9: Tin tức giao thông và tìm địa điểm lân cận.

(a) Hồ sơ cá nhân



(b) Lịch sử lộ trình



(c) Bản đồ ngoại tuyến

Hình 6.10: Hồ sơ, lịch sử lộ trình và bản đồ ngoại tuyến.

quyết định đường đi.

6.2.3. Chấm điểm sức khỏe tuyến A–F

Đọc theo mỗi tuyến ứng viên, ứng dụng lấy ~ 40 điểm mẫu, tra đoạn gần nhất mỗi điểm rồi đưa congestionIndex của chúng vào RouteQualityAnalyzer: tỉ lệ đoạn kẹt nặng/trung

bình quyết định câu trạng thái (“Có đoạn tắc nghẽn nặng — ước tính thêm ~25 phút” hoặc “Tuyến đường thông thoáng”) và điểm chữ A–F. Điểm số chỉ tính trên mẫu có dữ liệu thật, đoạn thiếu dữ liệu được ghi nhận riêng (NO_DATA) thay vì bị tính là thoáng.

6.2.4. Dự báo và gán cụm phía server (thẻ Dự báo ML)

Khi người dùng bấm vào một đoạn, đặc trưng được gửi tới `/api/ml/predict`; server dự đoán tốc độ bằng Random Forest, **gán cụm bằng chính mô hình K-Means $k = 6$ dùng chung với app** và trả về `clusterId` kèm `severityRank`. Người dùng nhìn thấy ngay “Cụm giao thông #2 — Rủi ro sự cố: THẤP” ngay dưới tốc độ live (Hình 6.6) — tức là *nhãn trạng thái do nghiên cứu tạo ra* được hiển thị trọn vẹn lên giao diện.

6.2.5. Tự động chuyển tuyến theo đề xuất ML

Với các tuyến có nhiều phương án, ứng dụng gửi đặc trưng mẫu dọc từng tuyến tới `/api/ml/recommend-route`. Server dự đoán tốc độ tại từng đoạn mẫu bằng Random Forest, quy đổi thành thời gian dự kiến toàn tuyến rồi chọn tuyến tốt nhất; nếu khác với tuyến ứng dụng đang hiển thị, app tự chuyển và thông báo “ML server đề xuất tuyến #k”. Nhờ đó mô hình dự đoán được huấn luyện lại với dữ liệu mới sẽ *tự động cải thiện* chất lượng gợi ý mà không cần thay đổi ứng dụng.

6.2.6. Lớp thành thật dữ liệu

Khoảng 12% đoạn trong tập huấn luyện rơi vào nhóm thiếu tín hiệu (tốc độ bằng 0). Ứng dụng kế thừa bài học này: đoạn mà TomTom không trả được dữ liệu live được tô **xám** “**thiếu dữ liệu**” thay vì mặc định xanh — tránh tạo ấn tượng sai rằng đường thoáng. Đây là ứng dụng trực tiếp của phân tích dữ liệu ở giai đoạn nghiên cứu vào thiết kế trải nghiệm.

6.2.7. Kịch bản người dùng cuối

Kịch bản 1 — sáng thứ Hai đi làm (giờ cao điểm): người dùng mở Trang chủ, bấm thẻ “Về công ty”. Trên bản đồ, các đoạn tuyến đường quen hiện đỏ/cam (cụm 2–5) trong khi các hướng vòng xanh (cụm 0–3); khi tìm đường, ứng dụng chấm điểm và chọn tuyến vòng dài hơn nhưng thông thoáng, kèm sức khỏe tuyến B thay vì F của tuyến thẳng. **Kịch bản 2 — chiều mưa lớn:** thời tiết xấu khiến tốc độ rơi; dữ liệu live cập nhật màu đoạn kẹt trong vài phút; người dùng bấm vào đoạn đỏ để xem thẻ Dự báo ML — tốc độ dự kiến sụt thêm và rủi ro sự cố tăng lên — rồi quyết định chọn hướng khác hoặc gọi xe.

6.3. Kiểm thử

6.3.1. Môi trường kiểm thử

Bảng 6.11 tóm tắt môi trường kiểm thử của đề tài.¹

Bảng 6.11: Môi trường kiểm thử.

Thành phần	Cấu hình
Ứng dụng	TrafficGo debug build, versionName 1.0 (minSdk 29, targetSdk 36)
Thiết bị	Trình giả lập MuMu Player (Android 15), adb qua TCP 16384
Server ML	Python 3.14, scikit-learn 1.8, FastAPI 0.141, Uvicorn trên localhost:8000
Dữ liệu	171 snapshot CSV, cluster_model.json (k=6), speed_rf.joblib
Công cụ	Gradle 8.11.1, JUnit 4 (unit test), adb, Swagger UI

6.3.2. Đo hiệu năng

Ngoài kiểm thử chức năng, hiệu năng của server ML được đo trực tiếp bằng lượt gọi REST thực tế trên máy phát triển (bảng 6.12): dự đoán một đoạn trung bình 67 ms (10 lần đo liên tiếp, bao gồm cả overhead mạng loopback), các endpoint trạng thái chỉ vài ms — thỏa mãn yêu cầu NFR01 (phản hồi ≤ 1 giây) với dữ liệu lớn; một request dự đoán tốn $\approx 0,07$ giây nghĩa là kể cả batch 100 đoạn vẫn hoàn tất trong ~ 1 giây khi chạy tuần tự (và nhanh hơn khi chạy song song nhờ async).

Bảng 6.12: Kết quả đo hiệu năng server ML (10 lần đo mỗi endpoint).

Endpoint	TB (ms)	Nhỏ nhất	Lớn nhất
POST /api/ml/predict	66	48	177
GET /api/ml/live-status	≈ 6	5	7
GET /health	≈ 5	5	6

6.3.3. Kiểm thử đơn vị

Bộ unit test chạy trên JVM gồm ba nhóm:

- (i) ClusterColorAssignerTest — nạp đúng cluster_model.json thật từ assets, kiểm tra ánh xạ cụm $\rightarrow$ thứ hạng $\rightarrow$ màu, xử lý đoạn thiếu dữ liệu;
- (ii) RouteQualityAnalyzerTest — xếp hạng sức khỏe tuyến A–F theo mẫu congestionIndex;
- (iii) PathFinderTest — dựng đồ thị và hệ số phạt kẹt (5.1).

¹Google Maps SDK yêu cầu thiết bị hỗ trợ OpenGL ES 2.0; các trình giả lập như MuMu/LDPlayer/BlueStacks hỗ trợ tốt, trong khi MEmu không — lỗi thường gặp khi bản đồ hiển thị trắng.

Kết quả: toàn bộ test đạt (BUILD SUCCESSFUL).

6.3.4. Kiểm thử chức năng

Bảng 6.13 tổng hợp các ca kiểm thử chức năng thực hiện trên trình giả lập; tất cả đều đạt sau khi các lỗi ở mục 6.3.5 được khắc phục.

Bảng 6.13: Kết quả kiểm thử chức năng trên trình giả lập.

Ca kiểm thử	Kết quả kỳ vọng / quan sát được	Kết quả
TC01 Đăng nhập	Firebase Auth xác thực thành công, chuyển tới Trang chủ	Đạt
TC02 Onboarding/Skip	Nút “Bỏ qua” đưa thẳng tới màn đăng nhập	Đạt
TC03 Nạp bản đồ	23.417 đoạn nạp trên luồng nền, không khựng UI	Đạt
TC04 Tô màu theo cụm	đoạn tô màu theo cụm, xám khi thiếu dữ liệu	Đạt
TC05 Bấm đoạn	Bottom sheet: tốc độ live, trạng thái, thẻ Dự báo ML	Đạt
TC06 Server offline	Thẻ báo “không khả dụng”, app không treo, tính năng khác bình thường	Đạt
TC07 Tìm đường	Routes API trả tuyến, vẽ polyline đúng, thống kê đầy đủ	Đạt
TC08 Cảnh báo tắc	Phát hiện đoạn tắc, cảnh báo “+25 phút”, sức khỏe F	Đạt
TC09 Đề xuất tuyến ML	Server đề xuất tuyến khác ⇒ app chuyển kèm thông báo	Đạt
TC10 Tin tức	RSS 5 báo, thumbnail, chip lọc danh mục	Đạt
TC11 Địa điểm lân cận	Places trả kết quả quanh GPS, lọc danh mục, khoảng cách đúng	Đạt
TC12 Lưu bản đồ ngoại tuyến	Chụp + nén nền, lưu file + metadata, hiện trong danh sách (1,2 MB)	Đạt
TC13 Hồ sơ	Cập nhật tên/phone/avatar, avatar hiển thị lại sau re-fresh	Đạt
TC14 Lịch sử lộ trình	Lưu tự động khi tuyến được xác nhận; “Trai lại tuyến này” hoạt động	Đạt
TC15 Đa ngôn ngữ	VI/EN áp dụng toàn app, giữ nguyên sau khởi động lại	Đạt
TC16 Gọi xe nhanh	Deep link mở Grab/Be/Xanh SM; chưa cài thì điều hướng CH Play	Đạt
TC17 Báo sự cố	Chọn loại sự cố tại tâm bản đồ, lưu Firebase, hiện marker cho người khác	Đạt
TC18 Chip trạng thái server	Hiển thị “đang hoạt động” khi server chạy; chuyển “ngoại tuyến” khi tắt server	Đạt

6.3.5. Lỗi phát hiện và khắc phục

Quá trình kiểm thử phát hiện và đã khắc phục khoảng 18 lỗi, tiêu biểu nhất:

- **Crash “Not on the main thread”:** gọi `getCameraPosition` của Maps SDK từ luồng nền khi lấy tâm camera để sắp thứ tự fetch — Maps SDK cấm gọi ngoài main thread. Khắc phục: chụp tâm camera ở main thread rồi truyền xuống luồng nền.

- **Tap đoạn không mở bottom sheet:** các polyline vẽ với `clickable=true` “nuốt” sự kiện chạm mà không có `OnPolylineClickListener`, khiến `onMapClick` không bao giờ kích hoạt khi chạm trúng đường. Khắc phục: đăng ký listener polyline và đọc segment từ tag.
- **Chạm bản đồ khi đang dẫn đường làm dờn đích:** guard `updateSelection` chỉ chặn bước 3, không chặn bước 4; camera khóa theo xe khiến chạm gần tâm màn hình dờn đích tới trước mũi xe vài chục mét và app báo nhầm “đã đến nơi”. Khắc phục: chặn từ bước 3 trở đi.
- **ANR nén PNG:** chụp bản đồ ngoại tuyến nén PNG 0,5–2 giây ngay trong callback trên main thread. Khắc phục: đẩy nén + ghi file xuống luồng nền, quay lại UI thread cho toast/finish.
- **OOM khi chọn ảnh đại diện:** decode full-resolution ảnh máy ảnh 12–50MP trước khi thu nhỏ 256px. Khắc phục: decode với `inSampleSize` nhắm 512px.
- **Cache traffic sống mãi:** `liveCongestionByIndex` không hết hạn khiến màu tuyến dùng dữ liệu hàng giờ tuổi; khắc phục bằng TTL 3 phút đồng bộ với bản đồ.
- **Fan-out TomTom mất kiểm soát:** zoom-out nhìn cả thành phố xếp hàng nghìn lời gọi API; khắc phục bằng ngưỡng zoom ≥ 14 và trần 150 đoạn/lần render ưu tiên gần tâm.
- **Callback muộn “hồi sinh” tuyến cũ:** sau `resetNavigation`, callback fetch trả về muộn vẫn vẽ lại tuyến đã hủy; khắc phục bằng generation counter.

6.4. Kết chương

Chương này đã trình bày hiện thực đầy đủ bốn thành phần của hệ thống với các kết quả thực nghiệm: K-Means $k = 6$ nhánh Feature Selection là mô hình gom cụm tốt nhất (Silhouette 0,4183); Random Forest dự đoán tốc độ đạt MAE 2,19 km/h và R^2 0,919 (cấu hình lite cho RAM 512 MB); ứng dụng TraffGo hiện thực trọn vẹn 14 yêu cầu chức năng với giao diện tiếng Việt thống nhất; 15 ca kiểm thử chức năng và toàn bộ unit test đạt sau khi khắc phục các lỗi phát hiện được.

CHƯƠNG 7

ĐÁNH GIÁ HỆ THỐNG

7.1. Đánh giá theo yêu cầu

Đối chiếu với Bảng 4.1 và 4.2: toàn bộ 14 yêu cầu chức năng đã được hiện thực và kiểm chứng (TC01–TC15); 7 yêu cầu phi chức năng đều đáp ứng — suy luận gom cụm chạy on-device, server phản hồi dưới 1 giây cho một request dự đoán, nạp 23.417 đoạn không khựng UI, app hoạt động đầy đủ khi không có server, khóa nhạy cảm tách khỏi git, mô hình chuẩn JSON dùng chung ba môi trường và giao diện đồng bộ một hệ thiết kế.

7.2. So sánh với các giải pháp hiện có

So với Google Maps (Bảng 3.2), hệ thống của đề tài có ba điểm khác biệt:

- (i) trạng thái giao thông được *phân cụm trên dữ liệu của chính TP.HCM* thay vì ngưỡng màu toàn cầu;
- (ii) người dùng nhìn thấy *cụm giao thông và dự báo ML* của từng đoạn, tạo tính minh bạch;
- (iii) bộ API ML mở cho phép tái sử dụng trong các hệ thống khác.

Đổi lại, độ phủ dữ liệu live của Google (đội xe khổng lồ) vẫn tốt hơn ở những vùng hệ thống chưa có snapshot — đây là hạn chế cần trung thực ghi nhận.

7.3. Ưu điểm

- Kết quả gom cụm được đưa tới người dùng cuối theo hai đường: nhúng on-device (nhanh, không cần mạng) và API server (dự đoán giám sát, phát hiện sự cố) — kết hợp linh hoạt với fallback an toàn.
- Đánh giá mô hình đa chỉ số, đa thuật toán trên dữ liệu thực tế; Feature Selection cải thiện Silhouette gấp 3,2 lần và thời gian pipeline chỉ ~70 giây, phù hợp tái huấn luyện định kỳ.
- Ứng dụng hoàn chỉnh và thực dụng: 20 màn hình, hai ngôn ngữ, đầy đủ từ bản đồ trạng thái đến tin tức, trợ lý giọng nói và ngoại tuyến.
- Server nhỏ gọn, tài liệu API tự sinh, quy trình tái huấn luyện một lệnh.

7.4. Hạn chế

- Gom cụm là pipeline offline; cập nhật dữ liệu mới đòi hỏi chạy lại toàn bộ (GMM/DBSCAN/Agglomerative tốn kém khi dữ liệu lớn).
- Dữ liệu thu thập gián tiếp qua API trung gian; chưa khai thác camera AI, cảm biến IoT tại nút giao thông nên độ trễ phản ánh tình huống tức thời còn hạn chế.
- Mô hình dự đoán tốc độ theo đoạn rời rạc, chưa mô hình hóa quan hệ không gian–thời gian giữa các đoạn (LSTM/GNN).
- API ML chưa có xác thực người gọi và rate-limit — phù hợp môi trường thử nghiệm nội bộ, chưa sẵn sàng công khai.
- Bộ dữ liệu thu trong ~5 tuần; tính khái quát theo mùa (mùa mưa, lễ Tết) chưa được kiểm chứng.

7.5. Phân tích rủi ro triển khai

Trước khi bàn giao, đề tài liệt kê các rủi ro vận hành chính kèm phương án ứng phó (bảng 7.1), giúp hệ thống không chỉ chạy được trong môi trường demo mà còn có lộ trình vận hành rõ ràng.

Bảng 7.1: Phân tích rủi ro triển khai và phương án ứng phó.

Rủi ro	Tác động	Phương án ứng phó
Hết quota TomTom (ngày)	Bản đồ thiếu dữ liệu live, màu chuyển xám	Xoay vòng 8 khóa tự động; giới hạn zoom ≥ 14 và trần 150 đoạn/lần render
Server ML offline	Không có dự báo/đề xuất từ server	Fallback on-device im lặng; chip trạng thái báo “ngoại tuyến”
Quy tắc Realtime Database siết chặt	Lỗi “Permission denied” khi tải địa điểm/ lịch sử	Công bố rules cho phép người dùng đã xác thực đọc/ghi nhánh Users/{uid} và incidents
Giới hạn Gemini free tier (20 yêu cầu/ngày)	Trợ lý giọng nói ngừng hiểu lệnh	Thông báo lỗi RESOURCE_EXHAUSTED rõ ràng; chỉ gọi khi người dùng chủ động bật mic
Model drift khi dữ liệu mới khác phân bố	Dự đoán lệch dần	Chạy lại train_model.py định kỳ; theo dõi MAE qua /health

7.6. Kiểm thử hồi quy sau khắc phục lỗi

Mọi lỗi được phát hiện trong quá trình kiểm thử (mục 6.3.5) sau khi khắc phục đều được *chạy lại toàn bộ các ca liên quan* để đảm bảo không tạo ra hồi quy (regression). Cụ thể: sau khi sửa lỗi crash luồng nền ở bản đồ, toàn bộ chuỗi TC03–TC06 được chạy lại ở cả hai trạng thái server (online/offline); sau khi sửa polyline click, luồng bottom sheet được kiểm tra lại ở ba mức zoom khác nhau; sau khi chuyển nén ảnh sang luồng nền, luồng lưu bản đồ ngoại tuyến chạy lặp ba lần liên tiếp để chắc chắn không rò rỉ tài nguyên. Kết quả: không phát hiện hồi quy nào, các ca kiểm thử đã sửa đều đạt ổn định qua nhiều lần chạy.

7.7. Kết chương

Hệ thống đáp ứng đầy đủ yêu cầu đặt ra với các ưu điểm rõ rệt về tính thực dụng và tính mở; các hạn chế còn lại đều đã được nhận diện cụ thể và gắn với hướng phát triển ở chương sau.

CHƯƠNG 8

TỔNG KẾT

8.1. Kết quả đạt được

- (1) Pipeline dữ liệu đa nguồn hoàn chỉnh: 171 snapshot (~ 170.000 dòng, 917 đoạn), join 100% dữ liệu thời tiết, chuẩn hóa và xuất tập đặc trưng phục vụ cả gom cụm lẫn dự đoán.
- (2) Đánh giá có hệ thống năm thuật toán gom cụm trên bộ chỉ số nội bộ lẫn ngoại bộ; mô hình tốt nhất là **K-Means** $k = 6$ **trên nhánh Feature Selection** (Silhouette 0,4183; DBI 0,8063; CH 147574,6) với sáu trạng thái giao thông có tính diễn giải rõ ràng.
- (3) Server Machine Learning FastAPI hiện thực đúng bộ 7 API thiết kế; mô hình **Random Forest** dự đoán tốc độ đạt **MAE 1,84 km/h**, R^2 **0,941** (baseline 8,25 km/h); kèm cơ chế phát hiện sự cố và gợi ý tuyến đường theo tốc độ dự đoán.
- (4) Ứng dụng **TraffiGo** (Android, Java, ~ 8.400 dòng code): bản đồ giao thông sáu mức theo cụm chạy on-device, định tuyến né kẹt với sức khỏe tuyến A–F, tin tức năm báo, địa điểm lân cận, gọi xe nhanh, trợ lý giọng nói, bản đồ ngoại tuyến, đa ngôn ngữ; rà soát và khắc phục ~ 18 lỗi tiềm ẩn.
- (5) Tích hợp hai chiều app $\leftrightarrow$ server kiểm chứng end-to-end trên trình giả lập; 15/15 ca kiểm thử chức năng và toàn bộ unit test đạt; build phát hành thành công.

Nhìn chung, đề tài đã đáp ứng các mục tiêu đặt ra ban đầu: từ dữ liệu đa nguồn, qua gom cụm có đánh giá chặt chẽ, tới sản phẩm người dùng cuối và dịch vụ dự đoán — tạo thành một vòng khép kín “dữ liệu $\rightarrow$ mô hình $\rightarrow$ ứng dụng”.

8.2. Hướng phát triển

- **Online/streaming clustering**: cập nhật cụm theo chu kỳ 5–15 phút thay vì chạy lại pipeline; nghiên cứu incremental K-Means.
- **Dữ liệu trực tiếp**: khai thác camera AI, cảm biến IoT, GPS đội xe thay vì chỉ API trung gian; giảm độ trễ phản ánh tình huống.
- **Mô hình không gian–thời gian**: LSTM/GNN/Transformer cho dự báo kẹt theo chuỗi thời gian và lan truyền trong mạng lưới; kết hợp dự báo với gom cụm (cụm $\rightarrow$ dự báo $\rightarrow$ cụm lại).

- **Bảo mật và vận hành API:** xác thực token, rate-limit, log tập trung, giám sát drift của mô hình khi triển khai công khai.
- **Mở rộng tính năng ứng dụng:** dự báo kẹt trước giờ cao điểm, cảnh báo proactive theo thói quen di chuyển, phiên bản web quản trị.

KẾ HOẠCH THỰC HIỆN

Đồ án tốt nghiệp được thực hiện trong 15 tuần (khoảng tháng 06 – tháng 09 năm 2026) theo kế hoạch Bảng 8.1. Trong đó, giai đoạn đầu (tuần 1–9) tập trung vào nghiên cứu, dữ liệu và mô hình học máy; giai đoạn sau (tuần 10–15) tập trung vào hiện thực sản phẩm, tích hợp hệ thống, kiểm thử và viết báo cáo. Kế hoạch được điều chỉnh linh hoạt theo tiến độ thực tế và ý kiến hướng dẫn của GVHD ở từng buổi họp nhóm.

Bảng 8.1: Kế hoạch thực hiện đồ án tốt nghiệp (15 tuần).

Tuần	Nhiệm vụ	Chi tiết
1–2	Khảo sát	Nghiên cứu công trình liên quan về gom cụm giao thông, khảo sát các nguồn dữ liệu và công nghệ nền tảng
3–4	Thu thập dữ liệu	Xây bộ công cụ thu snapshot giao thông TomTom, hạ tầng OSM/Mapbox, thời tiết Open-Meteo cho TP.HCM
5–6	Tiền xử lý	Làm sạch, khử trùng lặp, xử lý giá trị thiếu, phân tích phân bố và tương quan thuộc tính
7	Lựa chọn thuộc tính	Mutual Information, Random Forest Importance; đánh giá top-k và chốt tập thuộc tính
8–9	Gom cụm	K-Means, MiniBatchKMeans, DBSCAN, GMM, Agglomerative; tìm tham số; đánh giá đa chỉ số; diễn giải cụm
10	Ứng dụng mobile	Nhúng mô hình gom cụm, tô màu bản đồ, định tuyến né kẹt, hoàn thiện các tính năng cốt lõi
11–12	Server ML	Huấn luyện mô hình dự đoán tốc độ, xây dựng FastAPI, tích hợp hai chiều app ↔ server
13	Hoàn thiện app	Sửa lỗi, đồng bộ giao diện theo bộ thiết kế chung, tối ưu hiệu năng và dữ liệu live
14	Kiểm thử	Unit test + kiểm thử chức năng trên trình giả lập, khắc phục lỗi phát hiện được
15	Báo cáo	Viết báo cáo theo biểu mẫu, chuẩn bị slide và demo bảo vệ trước hội đồng

TÀI LIỆU THAM KHẢO

- [1] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Morgan Kaufmann Publishers, 2011.
- [2] J. MacQueen, “Some methods for classification and analysis of multivariate observations,” in *Proc. 5th Berkeley Symp. Math. Statist. and Prob.*, vol. 1, 1967, pp. 281–297.
- [3] S. Lloyd, “Least squares quantization in PCM,” *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, 1982.
- [4] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.
- [5] D. Reynolds, “Gaussian Mixture Models,” in *Encyclopedia of Biometrics*, vol. 741, 2009, pp. 659–663.
- [6] H. Peng, F. Long, and C. Ding, “Feature selection based on mutual information: Criteria of max-dependency, max-relevance, and min-redundancy,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 27, no. 8, pp. 1226–1238, 2005.
- [7] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [8] P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, 1987.
- [9] D. L. Davies and D. W. Bouldin, “A cluster separation measure,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-1, no. 2, pp. 224–227, 1979.
- [10] T. Caliński and J. Harabasz, “A dendrite method for cluster analysis,” *Communications in Statistics*, vol. 3, no. 1, pp. 1–27, 1974.
- [11] L. Hubert and P. Arabie, “Comparing partitions,” *J. Classification*, vol. 2, no. 1, pp. 193–218, 1985.
- [12] TomTom, “Traffic Flow Segment Data API,” 2026. [Online]. Available: <https://developer.tomtom.com>
- [13] OpenStreetMap Foundation, “OpenStreetMap and Overpass API,” 2026. [Online]. Available: <https://www.openstreetmap.org>
- [14] Open-Meteo, “Historical Weather API,” 2026. [Online]. Available: <https://open-meteo.com>

- [15] Google, “Maps SDK for Android, Routes API and Places API documentation,” 2026. [Online]. Available: <https://developers.google.com>
- [16] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [17] S. Tilkov and S. Vinoski, “Node.js: Using JavaScript to build high-performance network programs,” *IEEE Internet Comput.*, vol. 14, no. 6, pp. 80–83, 2010.

PHỤ LỤC A

HƯỚNG DẪN CHẠY SERVER ML

Server ML được đặt tại thư mục E:\Android\TraffiGo\server, gồm ba thành phần: `train_model.py` (huấn luyện và xuất mô hình), `app.py` (REST API) và thư mục `models/` (chứa các tệp mô hình). Các bước cài đặt và khởi chạy:

```
pip install -r requirements.txt
python train_model.py          # huấn luyện lại mô hình (chạy 1 lần)
uvicorn app:app --host 0.0.0.0 --port 8000
```

Sau khi khởi chạy, giao diện tài liệu API (Swagger UI) tự sinh tại `http://localhost:8000/docs`. Ứng dụng Android mặc định gọi `http://10.0.2.2:8000` (địa chỉ loop-back của máy phát triển nhìn từ trình giả lập chuẩn), sau đó tự thử `http://127.0.0.1:8000` (phù hợp khi dùng `adb reverse tcp:8000 tcp:8000` hoặc trình giả lập MuMu/LDPlayer); với thiết bị thật, người dùng đặt địa chỉ IP LAN của máy chủ trong `SharedPreferences` theo khóa `ml_server_url` (nhóm prefs `TraffiGoPrefs`). Khi máy chủ không khả dụng, mọi lời gọi ML tự fallback về logic tính toán trên thiết bị.

Máy chủ triển khai chính thức

Môi trường production của TraffiGo ML Server được triển khai trên Render Cloud tại `https://traffigo-server.onrender.com`. Đây là host mặc định ứng dụng Android ưu tiên kết nối; ứng dụng kiểm tra `https://traffigo-server.onrender.com/health` để xác định trạng thái server và hiển thị chip xanh “Server ML: đang hoạt động” khi kết nối thành công. URL localhost và các địa chỉ emulator ở trên chỉ phục vụ phát triển cục bộ; server production dùng HTTPS và biến môi trường PORT do nền tảng cung cấp.

PHỤ LỤC B

ĐẶC TRƯNG CÁC MÔ HÌNH

Nhóm gom cụm — 9 thuộc tính của nhánh Feature Selection:

- speedLimitRatio (tỉ lệ tốc độ hiện tại trên giới hạn tốc độ), crossTime (thời gian vượt đoạn), trafficVolume (lưu lượng ước lượng);
- congestionIndex (chỉ số ùn tắc), occupancy (độ chiếm dụng mặt đường), relative-CongestionIndex (chỉ số ùn tắc tương đối);
- freeFlowSpeed (tốc độ dòng tự do), lengthKm (chiều dài đoạn đường), curvatureIndex (chỉ số độ cong).

Nhóm dự đoán tốc độ — Random Forest:

- Thuộc tính số: freeFlowSpeed, speedLimit, hourOfDay, dayOfWeek, laneCount (số làn), intersectionCount (số giao lộ), routeSlopePercent (độ dốc), curvatureIndex, lengthKm;
- Thuộc tính thời tiết: precipitation (lượng mưa), windspeed (tốc độ gió), temperature (nhiệt độ cao nhất trong ngày);
- Thuộc tính phân loại (one-hot): roadType (loại đường), surface (bề mặt), dayType (loại ngày);
- Biến đích: currentSpeed.

Cả currentSpeed, congestionIndex và jamFactor đều **không** được dùng làm đầu vào của mô hình dự đoán vì chúng được suy ra trực tiếp từ biến đích (leakage), làm kết quả đánh giá mất tin cậy.